

Analisi della Manutenibilità e Predizione dei Difetti a Livello di Metodo: Un Approccio basato su Machine Learning e What-If Analysis

Flavio Simonelli

Matricola 0365333

Studente del Corso di Laurea Magistrale in Ingegneria Informatica
Università degli Studi di Roma "Tor Vergata"
flavio.simonelli@alumni.uniroma2.eu

ABSTRACT - Questo studio analizza la relazione tra le metriche di manutenibilità del codice e la propensione ai difetti nei sistemi software in evoluzione, con l'obiettivo di valutare l'efficacia del refactoring mirato nella riduzione della probabilità di bug. L'approccio adottato per la scelta del modello prevede l'implementazione di una pipeline di machine learning basata sulla validazione walk-forward applicata ai dataset dei progetti Apache Bookkeeper e OpenJPA, integrando tecniche di feature selection (come InfoGain e BestFirst-CFS) e bilanciamento dei dati (Smote, UnderSampling). I risultati delle analisi di correlazione di Spearman hanno permesso di identificare la metrica *LOC* come il principale indicatore di criticità. Attraverso un'analisi "What-If", è stato stimato che l'applicazione di refactoring mirata sulla metrica *N_{Smell}* ha portato a una riduzione del 29% della probabilità di difetti prevista dal classificatore RandomForest. Questi risultati suggeriscono che la prioritizzazione degli interventi di manutenzione basata su modelli predittivi può mitigare efficacemente l'insorgenza di bug futuri, ottimizzando gli sforzi di sviluppo.

1 Introduzione

1.1 Contesto e Motivazione

Nel moderno panorama dell'ingegneria del software, la manutenibilità e la propensione ai difetti rappresentano fattori critici per il successo a lungo termine di un progetto. Con la continua evoluzione del codice, l'accumulo di debito tecnico può degradare progressivamente la qualità del sistema, rendendo i componenti software più vulnerabili all'insorgenza di bug. Sebbene la relazione teorica tra metriche di processo e qualità del software sia stata ampiamente esplorata, la capacità di prevedere con precisione quali metodi diventeranno difettosi rimane una sfida aperta, specialmente nel contesto di cicli di rilascio rapidi e iterativi.

1.2 Definizione del Problema

Sebbene la letteratura sia ampia, molti studi analizzano gli smell a un livello di granularità elevato (classi o pacchetti). Questa focalizzazione rende difficile per gli sviluppatori individuare con precisione la posizione dei bug e pianificare

refactoring puntuali. Il presente studio mira a superare tale limite analizzando i dati storici dei progetti Apache Bookkeeper e OpenJPA. Attraverso tecniche di Machine Learning, l'obiettivo è correlare gli smell a livello di singolo metodo alla probabilità di difettosità, fornendo una guida pratica per le decisioni di manutenzione.

1.3 Domande di Ricerca

L'obiettivo principale della ricerca è valutare come l'analisi dei dati storici e l'applicazione di modelli predittivi possano supportare un processo di manutenzione preventiva basato sui dati. Nello specifico, lo studio risponde alle seguenti domande:

- **RQ1 (Valutazione dei Predittori):** Quale classificatore presenta il grado di accuratezza superiore tra quelli analizzati?
- **RQ2 (Efficacia del Refactoring):** In che misura l'intervento di refactoring sul metodo più critico (scelto in base alla metrica più correlata al bug nell'ultima release) ha influenzato le feature correlate alla *bugginess*?
- **RQ3 (Analisi What-If):** Quanti metodi difettosi sarebbero potuti essere evitati eliminando completamente gli *smell* di codice (scenario *zero-smells*)?

1.4 Struttura del Report

Il documento è organizzato come segue: la Sezione 2 descrive la metodologia di misurazione, il dataset e il protocollo sperimentale; la Sezione 3 presenta i risultati quantitativi, l'analisi comparativa del refactoring e la stima *What-If*; la Sezione 4 discute le implicazioni dei risultati e le minacce alla validità dello studio; infine, la Sezione 5 conclude il lavoro delineando possibili sviluppi futuri. Il codice sorgente sviluppato per questo studio è disponibile pubblicamente nel repository GitHub [1]. Contestualmente, l'analisi della qualità del codice e il monitoraggio delle metriche di manutenibilità sono consultabili sulla piattaforma SonarCloud [2].

2 Misurazioni e Metodologia

Questa sezione illustra la metodologia adottata per la costruzione del dataset e la definizione delle metriche, de-

scrivendo il processo automatizzato che interfaccia le API di Jira con il repository Git del progetto.

2.1 Costruzione del Dataset

Il processo di estrazione dati utilizza le release registrate su Jira come punto di riferimento temporale primario. Per garantire la coerenza delle analisi, sono state estratte tutte le versioni disponibili, applicando un filtro di pulizia che elimina le release indicate come "non rilasciate" o prive di una data di rilascio ufficiale.

I dati relativi ai difetti derivano dai ticket *Jira* classificati come *Bug* (stato *Resolved/Closed*, risoluzione *Fixed*). Per assicurare l'integrità del dataset, sono state scartate le occorrenze con incongruenze logiche, filtrando specificamente i ticket privi di *Fixed Version* (FV) o con una *Affected Version* (AV) cronologicamente successiva alla FV.

Parallelamente, il codice sorgente è stato estratto tramite la libreria JGit. Per ogni release R_i , è stato ricostruito lo snapshot del sistema posizionandosi sull'ultimo commit effettuato entro la data di rilascio di R_i (estratta da Jira). Durante l'analisi dei file, sono stati applicati filtri per escludere artefatti non pertinenti alla produzione, scartando i percorsi di test (es. `/test/`) e la documentazione, limitando l'estrazione ai soli file con estensione `.java`.

2.2 SZZ e Labeling

Una sfida critica nell'identificazione dei metodi difettosi riguarda la mancanza del dato sulla *Injected Version* (IV) in molti ticket Jira.

2.2.1 Algoritmo SZZ Proportion

Nei casi in cui un ticket possiede la *Fixed Version* ma manca della *Injected Version*, è stata applicata l'euristica della **Proportion**. La versione di iniezione (IV) viene stimata secondo la formula:

$$IV = FV - (P \times (FV - OV)) \quad (1)$$

Dove:

- FV è la Fixed Version (versione del fix).
- OV è la Opening Version (versione in cui il ticket è stato aperto).
- P è il fattore di proporzione.

Il calcolo di P è stato eseguito utilizzando un approccio **Moving Window**. Il valore di P per un dato bug viene calcolato come la media delle proporzioni osservate nei ticket precedenti che possedevano una *Injected Version* valida. La dimensione della finestra è stata impostata all'1% del numero totale di ticket estratti aventi *Affected Version* presente, garantendo che la stima si adatti all'evoluzione temporale del progetto e non sia affetta dal fenomeno di *data leakage*.

2.2.2 Linking e Labeling

L'identificazione dei metodi difettosi è avvenuta analizzando i messaggi di commit: sono stati considerati *fix-commit* tutti i commit contenenti l'ID del ticket nel messaggio. Dato che un bug può essere risolto tramite molteplici commit, l'analisi copre l'intero intervallo temporale dalla Opening Version alla Fix Version. Sono state

etichettate come *Buggy* tutte le istanze dei metodi toccati dai fix-commit nelle versioni comprese nell'intervallo $[IV, FV)$, escludendo la versione di fix stessa (in cui il difetto è, per definizione, risolto).

2.3 Metriche e Definizioni

Per caratterizzare in modo esaustivo il profilo di ogni metodo, sono state considerate un totale di **31 metriche**. La selezione include sia metriche *actionable* (come *NSmells*, che possono essere direttamente influenzate tramite operazioni di refactoring) sia metriche *non-actionable* (utili per la predizione ma non modificabili a posteriori, come la *NR*).

Le metriche si dividono in due macro-categorie:

- **Metriche Statiche:** Calcolate analizzando lo snapshot del codice al momento del rilascio.
- **Metriche Temporal:** Derivate dall'analisi della cronologia dei commit.

Per catturare le dinamiche evolutive del software, le metriche temporali non vengono considerate solo nella loro versione base, ma sono state sdoppiate per contestualizzare l'informazione:

1. **Versione Locale:** Misura le variazioni avvenute esclusivamente nell'intervallo tra la release precedente (R_{i-1}) e quella corrente (R_i).
2. **Versione Globale:** Accumula i valori dall'introduzione del metodo nel progetto fino alla release corrente, fornendo una visione storica completa.

Inoltre, per le metriche quantitative in cui la distribuzione dei valori può essere significativa (es. *Churn* o *LOC-Added*), sono state calcolate anche le aggregazioni **MAX** (picco massimo in un singolo commit) e **AVG** (media per revisione). Questo approccio granulare permette ai modelli di machine learning di discernere se la difettosità è correlata a un volume totale di modifiche elevato o a singoli eventi traumatici nello sviluppo (picchi di complessità). La Tabella 1 riporta l'elenco completo delle feature estratte.

2.3.1 Metriche di Valutazione dei Classificatori

Per confrontare le performance dei predittori (RQ1) e selezionare il modello ottimale per l'analisi "What-If", sono state adottate le seguenti metriche di valutazione:

- **Precision:** La proporzione di istanze classificate come *Buggy* che sono realmente difettose, indicando l'affidabilità del modello nel minimizzare i falsi positivi.
- **Recall:** La frazione di metodi realmente difettosi che vengono correttamente identificati dal modello, misurando la capacità di copertura rispetto ai falsi negativi.
- **F1-Score:** La media armonica tra Precision e Recall, utilizzata per fornire una misura sintetica che bilancia il trade-off tra accuratezza e copertura.
- **AUC (Area Under the Curve):** La probabilità che il classificatore assegni un punteggio di rischio più alto a un'istanza *Buggy* scelta casualmente rispetto a una *Clean*, valutando la capacità di separazione delle classi indipendentemente dalla soglia di decisione.

- **Kappa (Cohen's Kappa):** Un coefficiente statistico che misura l'accordo tra la classificazione predetta e quella reale, normalizzando l'accuratezza rispetto alla probabilità di un accordo dovuto al caso.
- **NPofB20:** Una metrica *effort-aware* che quantifica il numero di bug reali individuati ispezionando il top 20% delle linee di codice ordinate per probabilità di difetto decrescente.

2.3.2 Analisi di Correlazione

Al fine di identificare le feature più critiche e guidare la selezione del metodo da rifattorizzare, viene utilizzata la **Correlazione di Spearman** (ρ). A differenza della correlazione di Pearson, Spearman valuta la relazione monotonica tra le variabili basandosi sui ranghi, risultando più robusta agli outlier e in grado di catturare associazioni non lineari tra le metriche software e la presenza di bug.

2.4 Protocollo Sperimentale

La validazione dei modelli segue un protocollo **Walk-forward** eseguito 10 volte come previsto dalla letteratura, progettato per simulare rigorosamente uno scenario reale di rilascio ed evitare fenomeni di *data leakage* temporale. Ad ogni iterazione, il modello viene addestrato sulle release passate e testato esclusivamente sulla release corrente.

2.4.1 Snoring e Data Cleaning

Per mitigare il fenomeno del *code snoring* (difettosità latente che introduce rumore), è stato applicato un taglio netto delle release finali. Nello specifico, il dataset è stato depurato scartando l'ultimo **66%** delle release per entrambi i progetti analizzati.

2.4.2 Data Balancing

Data la natura intrinsecamente sbilanciata dei dataset di difetto (dove le classi *Clean* sovrastano numericamente le classi *Buggy*), sono state applicate tecniche di campionamento per prevenire il bias del classificatore verso la classe maggioritaria:

- **SMOTE (Synthetic Minority Over-sampling Technique):** Applicato al progetto *Bookkeeper* per generare istanze sintetiche della classe minoritaria (Buggy) tramite interpolazione.
- **Under-sampling:** Applicato al progetto *OpenJPA* per ridurre casualmente le istanze della classe maggioritaria (Clean).

La scelta differenziata è stata guidata dalla dimensione volumetrica dei dataset grezzi, bilanciando la necessità di informazioni sintetiche con il costo computazionale.

2.4.3 Feature Selection e Analisi del Bias

Al fine di identificare il miglior set di predittori e valutare l'impatto del *data leakage* sulle performance, il processo di Feature Selection è stato condotto secondo due modalità distinte e confrontate tra loro:

1. **Global Feature Selection:** La selezione viene eseguita *una tantum* sull'intero dataset prima della validazione. Sebbene computazionalmente efficiente, questo approccio è stato dimostrato introdurre un *bias di selezione* da **Ambroise e McLachlan**

(2002) [3]. Tale fenomeno è stato ulteriormente analizzato da **Cawley e Talbot (2010)** [4].

2. **Walk-forward Feature Selection:** La selezione viene eseguita dinamicamente ad ogni iterazione (fold) del protocollo walk-forward, utilizzando esclusivamente i dati di training disponibili al momento della release corrente. Questo approccio implementa rigorosamente il protocollo di validazione "interno" raccomandato da [4].

Per entrambe le modalità, sono state applicate due tecniche di tipo **Filter** per confrontare strategie di selezione diverse:

- **Ranking Filter (InfoGain):** Selezione statica delle migliori 10 feature (Top-k) basata sull'Information Gain, che valuta ogni attributo singolarmente.
- **Subset Filter (BestFirst + CFS):** Combinazione dell'algoritmo di ricerca *greedy* (BestFirst) con il valutatore *Correlation-based Feature Selection* (CFS). A differenza del ranking, questo metodo valuta il potere predittivo di interi sottoinsiemi di feature, selezionando quello che massimizza la correlazione con la classe target minimizzando la ridondanza tra le feature stesse (multicollinearità).

2.5 Assunzioni e Limitazioni

Al fine di rendere lo studio replicabile, gestire la complessità computazionale e mitigare potenziali distorsioni, sono state adottate le seguenti assunzioni e semplificazioni metodologiche:

- **Proportion Cold Start ($P = 1$):** Nella fase iniziale dell'algoritmo di stima della *Injected Version*, quando la finestra mobile storica è vuota, è stata adottata l'euristica $P = 1$. Questo assume che il bug sia stato introdotto esattamente nella versione di apertura del ticket ($IV = OV$), rappresentando una stima conservativa in assenza di dati storici.
- **Selezione e Tuning dei Classificatori:** L'indagine è stata circoscritta a tre algoritmi standard (*Naive Bayes*, *Random Forest*, *IBk*) utilizzando le configurazioni di default, senza esplorare modelli complessi o eseguire ottimizzazione degli iperparametri.
- **Strategia di Bilanciamento Rigida:** L'applicazione di SMOTE e Under-sampling mira a ottenere una parità esatta (1:1) tra le classi nel training set, senza trattare il *sampling ratio* come iperparametro variabile.
- **Esclusione di Feature Identificative:** L'attributo *releaseID*, pur presente nel dataset grezzo per l'ordinamento cronologico, è stato escluso dal set. Si assume che tale metrica non apporti reale valore predittivo sulla qualità del codice, ma rischi di introdurre un *bias* temporale, inducendo il modello a basare le predizioni sulla sequenza delle release piuttosto che sulle caratteristiche strutturali del software.
- **Accuratezza dell'Analisi Statica:** Per il calcolo della metrica *NSmells* è stato utilizzato lo strumento PMD. Nell'interpretazione dei risultati, si assume e

si accetta il margine di errore intrinseco degli strumenti di analisi statica automatizzata; è noto infatti che PMD è soggetto a un tasso di falsi positivi stimato attorno al 5-10%, che potrebbe introdurre un lieve rumore.

3 Risultati

3.1 Valutazione delle Performance Predittive (RQ1)

L'analisi dei risultati empirici (dettagliati nell'Appendice A) dimostra che non esiste un classificatore universalmente superiore per tutte le metriche e in tutti i contesti. Emerge piuttosto la necessità di valutare le prestazioni caso per caso, isolando le specificità di ogni progetto software. Di seguito vengono discussi separatamente i risultati per Apache Bookkeeper e Apache OpenJPA. Sono disponibili i grafici risultanti dalle valutazioni corrispettivamente in Appendice A.2 e A.3 e i valori aggregati in Tabella 6 e Tabella 11.

3.1.1 Apache Bookkeeper

Feature Selection e Concept Drift Un confronto tra le strategie di selezione ha mostrato che l'applicazione della *Feature Selection* in modalità *Per-Fold* produce risultati lievemente superiori rispetto alla modalità *Global*, specialmente utilizzando l'algoritmo *BestFirst*. Questo fenomeno è attribuibile all'evoluzione temporale del software: la rilevanza delle metriche predittive varia nel tempo (*concept drift*) e l'approccio *Per-Fold*, adattandosi ai dati di training specifici di ogni iterazione, cattura queste variazioni meglio di una selezione statica globale. Tuttavia, analizzando l'approccio basato su *Information Gain* in modalità globale, si è riscontrato un leggero bias positivo (*data leakage*), dovuto al fatto che il calcolo del guadagno informativo sull'intero dataset anticipa impropriamente informazioni relative ai fold di test.

Nonostante ciò, le tecniche di selezione non hanno offerto vantaggi sostanziali rispetto alla configurazione base. La *Forward Search* tende a convergere prematuramente su minimi locali a causa dell'alta dimensionalità dello spazio delle metriche, selezionando sottoinsiemi di feature insufficienti. Empiricamente, la configurazione **NoSelection** ha garantito i risultati migliori in termini di AUC, Kappa e F-Measure.

Bilanciamento e Scelta del Modello L'uso di *SMOTE* ha confermato le attese teoriche innalzando la *Recall*, ma con incrementi marginali a fronte di un decadimento della *Precision*. Dato che il beneficio netto è risultato limitato, il costo computazionale aggiuntivo non è stato ritenuto giustificabile. Tra i classificatori, *Random Forest* (pur con prestazioni simili a *IBk*) si è confermato il modello più robusto in termini di stabilità delle metriche AUC, Kappa e NPofB20. Pertanto, privilegiando l'efficienza e la precisione, per Bookkeeper è stata selezionata la configurazione: **Random Forest, NoSelection e NoSampling**.

3.1.2 Apache OpenJPA

L'analisi su OpenJPA ha confermato parzialmente i trend di Bookkeeper, ma con differenze marcate nell'impatto delle tecniche di preprocessing.

Feature Selection e Bias Anche in questo caso, il confronto tra l'approccio *BestFirst* globale e quello *Per-Fold* ha reso evidente il bias di selezione introdotto dall'analisi sull'intero dataset. Come per Bookkeeper, l'approccio conservativo (*NoSelection*) si è rivelato preferibile.

Impatto dell'Undersampling A differenza di Bookkeeper, su OpenJPA l'uso dell'*Undersampling* ha avuto un impatto drastico. Si osserva un netto trade-off: l'eliminazione della classe maggioritaria innalza significativamente la *Recall* (capacità di trovare bug), ma abbassa la *Precision*. La scelta della configurazione diventa quindi dipendente dallo scenario: in contesti *safety-critical* è preferibile l'Undersampling per minimizzare i falsi negativi; in contesti a basso rischio, l'approccio senza sampling garantisce che le segnalazioni di bug siano molto affidabili (alta Precision).

Scelta del Modello Finale Confrontando *IBk* e *Random Forest*, quest'ultimo prevale leggermente grazie a una migliore precisione a parità di altre metriche. Per le successive analisi *what-if* sugli smell, si è deciso di adottare per OpenJPA la configurazione: **Random Forest, NoSelection e NoSampling**.

3.2 Efficacia del Refactoring (RQ2)

Per entrambi i progetti, la selezione del candidato al refactoring è avvenuta individuando il metodo classificato come *buggy* nell'ultima release analizzata che presentasse il valore più alto di LOC. Tale criterio deriva dall'analisi di correlazione di Spearman (Tabelle 2 e 7), che ha indicato la dimensione del codice come la metrica *actionable* più fortemente correlata alla difettosità ($\rho \approx 0.11$ per Bookkeeper, $\rho \approx 0.23$ per OpenJPA), a fronte di metriche di processo storiche non modificabili. L'intervento, eseguito applicando il pattern *Extract Method*, ha prodotto risultati differenti in base all'interazione tra le feature. In `org.apache.bookkeeper.benchmark.BenchThroughputLatency.main(String[])` (Tabelle 3 e 4), la decomposizione ha determinato un abbattimento netto delle feature correlate *positivamente* con i difetti: la Complessità Ciclomantica è scesa da 16 a 6 e i Code Smell da 17 a 3. Poiché l'analisi non ha evidenziato feature correlate *negativamente* (fattori di protezione) significative su cui fare leva, il successo è dipeso interamente dalla minimizzazione delle metriche "tossiche", sufficiente a invertire la predizione di rischio da Buggy a **Clean** nonostante l'aumento fisiologico del *churn*. In `org.apache.openjpa.persistence.jdbc.AnnotationPersistenceMappingParser.parseMemberMappingAnnotations(FieldMetadata)` (Tabelle 8 e 9), la riduzione delle feature strutturali positivamente correlate (LOC, Nesting Depth e Complessità Ciclomantica, crollata da un picco di 57 a un massimo di 11) ha bonificato efficacemente 14 dei 15 metodi derivati (predetti **Clean**). Tuttavia, nel metodo residuo M5, il *trade-off* tra feature positive è risultato sfavorevole: la riduzione delle metriche statiche (LOC da 185 a 22) non è bastata a compensare l'incremento delle metriche di processo anch'esse correlate positivamente alla bugginess, come *Global Churn* ($\rho = 0.169$) e *Global LOC Added* ($\rho = 0.181$). La persistenza di queste ultime ha mantenuto la predizione a **Buggy**, evidenziando come in assenza di feature correlate negativamente che fungano da "cuscinetto", l'aumento delle

metriche di processo post-refactoring possa mascherare i benefici strutturali.

3.3 Impatto della Prevenzione degli Smell (RQ3)

Per quantificare l'impatto specifico dei code smell sulla probabilità di difettosità dei metodi, è stata condotta un'analisi *what-if* controfattuale basata sui predittori ottimali selezionati nella RQ1. L'esperimento simula uno scenario "Zero-Smells" attraverso la segmentazione del dataset in tre sottoinsiemi logici:

- **Dataset B^+ :** Il sottoinsieme dei metodi che presentano almeno uno smell ($NSmells > 0$).
- **Dataset B_{fixed} :** Una variante sintetica di B^+ , in cui la feature $NSmells$ è stata forzata a zero, mantenendo inalterate le altre metriche di processo e complessità.
- **Dataset C :** Il sottoinsieme di metodi privi di smell ($NSmells = 0$), i cui guasti sono imputabili a cause diverse e quindi non prevenibili tramite la rimozione degli smell.

Analisi su Apache Bookkeeper In riferimento alla Tabella 5, il dataset B^+ conta inizialmente 2.755 metodi difettosi correlati a smell. Applicando il modello predittivo sullo scenario simulato B_{fixed} , il numero atteso di metodi difettosi scende a 1.951. La differenza di **804** metodi ($2.755 - 1.951$) rappresenta la quota di difetti che sarebbe stato possibile evitare eliminando completamente gli smell. In termini relativi, ciò corrisponde a una riduzione del **29,18%** sul sottoinsieme dei metodi affetti da smell (B^+). Considerando invece il totale complessivo dei difetti nel progetto (Dataset A = 6.790), l'intervento di bonifica degli smell avrebbe comportato una riduzione globale del **11,84%**, tenendo conto che 4.035 difetti (Dataset C) sono strutturalmente indipendenti dalla presenza di smell.

Analisi su Apache OpenJPA Per il progetto OpenJPA (Tabella 10), partendo da un sottoinsieme B^+ di 24.752 metodi difettosi, la simulazione dello scenario *clean* riduce le predizioni di difettosità a 17.696 unità. L'intervento di refactoring avrebbe quindi prevenuto **7.056** difetti. Questo valore si traduce in una riduzione del **28,51%** rispetto al sottoinsieme trattabile (B^+). Proiettando questo risultato sul totale assoluto dei difetti registrati (Dataset A = 53.604), la rimozione totale degli smell avrebbe garantito un abbattimento complessivo della difettosità pari al **13,16%**, al netto dei 28.852 bug (Dataset C) non correlati agli smell.

4 Discussioni e minacce

- **RQ1:** Non è emerso un classificatore universalmente superiore, tuttavia **Random Forest** si è distinto come il modello più robusto in entrambi i progetti. La configurazione ottimale, capace di massimizzare stabilità e precisione, è risultata essere **Random Forest senza Feature Selection né Sampling**. Le tecniche di selezione hanno mostrato tendenze al bias o alla convergenza prematura, mentre le tecniche di bilanciamento (SMOTE/Undersampling) hanno compromesso eccessivamente la precisione.
- **RQ2:** L'intervento di refactoring, guidato dalla metrica *actionable* più correlata (LOC) ed eseguito tra-

mite *Extract Method*, ha avuto successo nel ridurre le feature strutturali correlate positivamente.

In **Apache Bookkeeper**, la riduzione delle metriche statiche è stata sufficiente a invertire la predizione da *Buggy* a *Clean*.

In **Apache OpenJPA**, si è ottenuta una **compartimentazione del rischio**: 14 su 15 metodi derivati sono risultati *Clean*. Un solo metodo è rimasto *Buggy* poiché l'incremento delle metriche di processo (Churn) generato dal refactoring ha sovrastato i benefici della semplificazione strutturale.

- **RQ3:** La simulazione dello scenario *zero-smells* ha evidenziato un elevato potenziale preventivo. L'eliminazione totale degli smell avrebbe evitato circa il **29%** dei difetti nel sottoinsieme di metodi affetti da smell, corrispondente a una riduzione globale della difettosità del progetto stimata tra l'**11%** e il **13%**.

L'analisi è soggetta a minacce alla validità interna, tra cui errori nella misura automatica delle metriche e assunzioni semplificative nell'esperimento *what-if*, che presume indipendenza tra le feature e la possibilità di portare a 0 qualsiasi feature *actionable*.

5 Conclusioni e Sviluppi Futuri

Dai risultati emergono due implicazioni chiave per la manutenzione del software:

1. **Sensibilità al Processo:** Il refactoring migliora la qualità statica ma incrementa il rischio a breve termine alzando il livello di *Churn*; i modelli predittivi devono saper bilanciare il beneficio strutturale con il "costo" di processo per non generare falsi allarmi.
2. **Prevenzione vs Cura:** Circa un terzo dei bug legati alla qualità del codice è strutturalmente prevenibile tramite la rimozione tempestiva degli smell, confermando il debito tecnico come causa concreta e misurabile di difettosità.

I prossimi passi futuri potrebbero includere:

- **Analisi Longitudinale Post-Refactoring:** Studiare l'evoluzione temporale dei metodi rifattorizzati per determinare il "tempo di assestamento" necessario affinché l'aumento del *churn* smetta di influenzare negativamente la predizione di rischio.
- **Estensione della Validità Esterna:** Replicare lo studio su un corpus eterogeneo di progetti, inclusi linguaggi diversi da Java.
- **Automazione in CI/CD:** Integrare il modello *Random Forest* ottimizzato in pipeline di Continuous Integration, per fornire agli sviluppatori un feedback in tempo reale sull'impatto (positivo o negativo) delle modifiche strutturali appena committate.
- **Labeling Assistito da ML:** Applicare tecniche di Machine Learning per analizzare semanticamente i commit di correzione (fix) e i report dei ticket. L'obiettivo è predire con maggiore accuratezza la versione esatta di iniezione del difetto (*bug injection*), superando i limiti di precisione degli attuali approcci euristici (es. algoritmo SZZ) e migliorando la qualità del dataset di training.

Riferimenti bibliografici

- [1] Flavio Simonelli. ProjectAnalyzerAI GitHub Repository. <https://github.com/flavio-simonelli/ProjectAnalyzerAI>, 2026.
- [2] Flavio Simonelli. SonarCloud analysis for ProjectAnalyzerAI. https://sonarcloud.io/project/overview?id=flavio-simonelli_ProjectAnalyzerAI, 2026.
- [3] Christophe Ambroise and Geoffrey J. McLachlan. Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences*, 99(10):6562–6566, 2002.
- [4] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 2010.
- [5] B. Vandehei, D. A. Da Costa, and D. Falessi. Leveraging the Defects Life Cycle to Label Affected Versions. *ACM Transactions on Software Engineering and Methodology*, 30:35, 2021.
- [6] D. Falessi, S. Mesiano Laureani, J. Çarka, M. Esposito, and D. A. Da Costa. Enhancing the defectiveness prediction of methods and classes via JIT. *Empirical Software Engineering*, 28(37):43, 2023.
- [7] J. Çarka, D. Falessi, and M. Esposito. On effort-aware metrics for defect prediction. *Empirical Software Engineering*, 27(152):38, 2022.
- [8] D. Falessi, L. Narayana, J. F. Thai, and B. Turhan. Preserving Order of Data When Validating Defect Prediction Models. page 20. Technical Report / Preprint.
- [9] D. Falessi, B. Russo, and M. Kathleen. What if I had no smells? In *ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, page 17. IEEE/ACM, 2017.

A Appendice

A.1 Dettaglio Metriche

Tabella 1: Dettaglio delle 31 Metriche con Scope e Aggregazioni

Metrica	Tipo	Scope		Aggregazioni			Descrizione
		Loc	Glob	Sum	Max	Avg	
LOC	Statica	-	-	-	-	-	Linee di codice fisiche.
StatementCount	Statica	-	-	-	-	-	Istruzioni eseguibili.
ParameterCount	Statica	-	-	-	-	-	Numero di parametri.
LocalVarCount	Statica	-	-	-	-	-	Variabili locali dichiarate.
Cyclomatic	Statica	-	-	-	-	-	Complessità di McCabe.
Cognitive	Statica	-	-	-	-	-	Complessità Cognitiva.
NestingDepth	Statica	-	-	-	-	-	Profondità annidamento.
ReturnComp	Statica	-	-	-	-	-	Complessità tipo di ritorno.
NSmells	Statica	-	-	-	-	-	Violazioni PMD.
NR	Processo	✓	✓	✓	-	-	Numero di revisioni (commit).
NAuth	Processo	✓	✓	✓	-	-	Numero autori unici.
LOC_Added	Processo	✓	✓	✓	✓	✓	Linee di codice aggiunte.
LOC_Deleted	Processo	✓	✓	✓	✓	✓	Linee di codice rimosse.
Churn	Processo	✓	✓	✓	✓	✓	Turbolenza (Added + Deleted).

Legenda: Loc = Locale (intervallo corrente), Glob = Globale (storico accumulato), Sum = Somma totale, Max = Picco massimo, Avg = Media per release.

A.2 Apache Bookkeeper

Figura 1: Distribuzione dell'AUC con feature Selection Globale - Apache Bookkeeper.

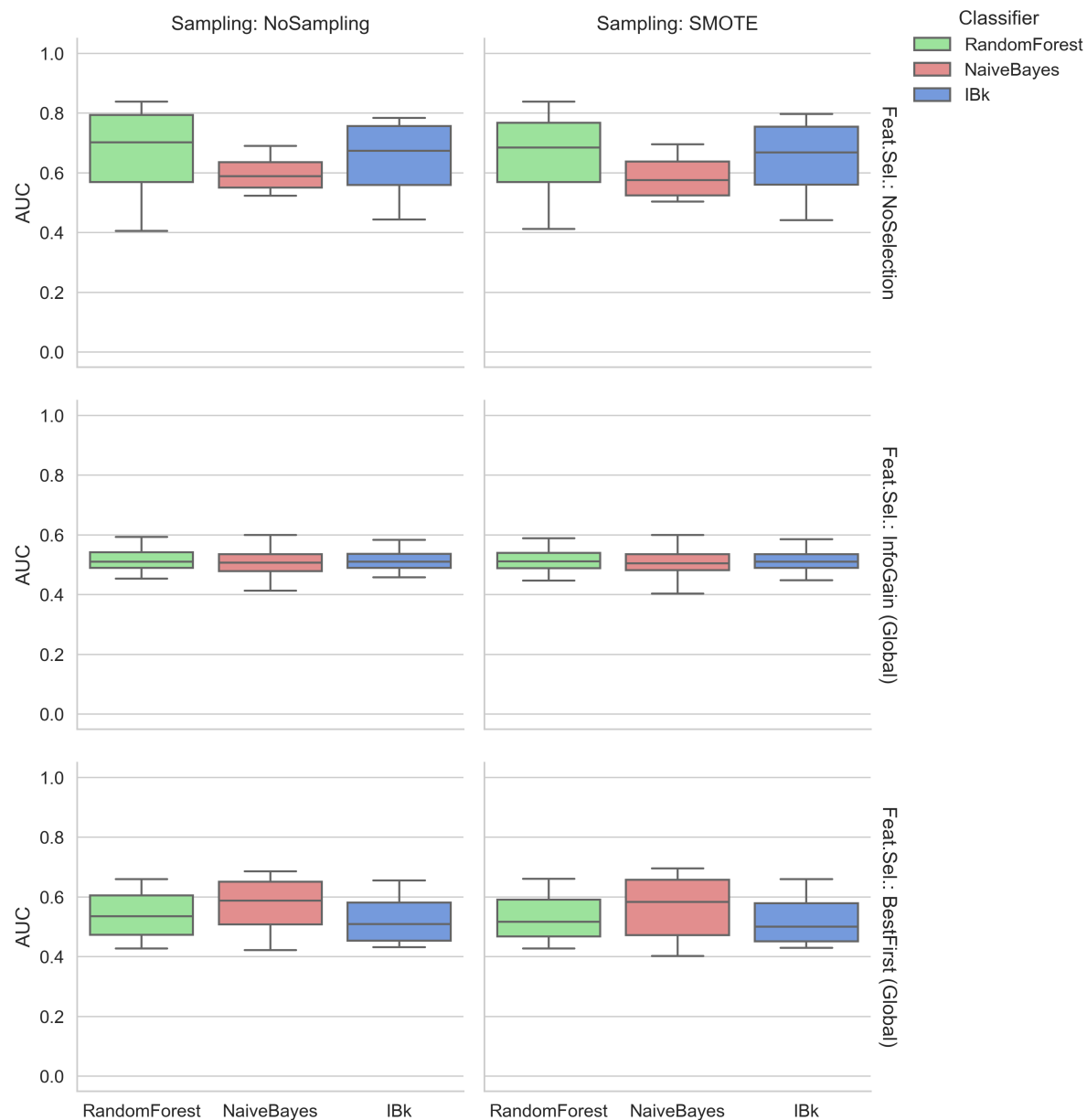


Figura 2: Distribuzione della F-Measure con feature Selection Globale - Apache Bookkeeper.

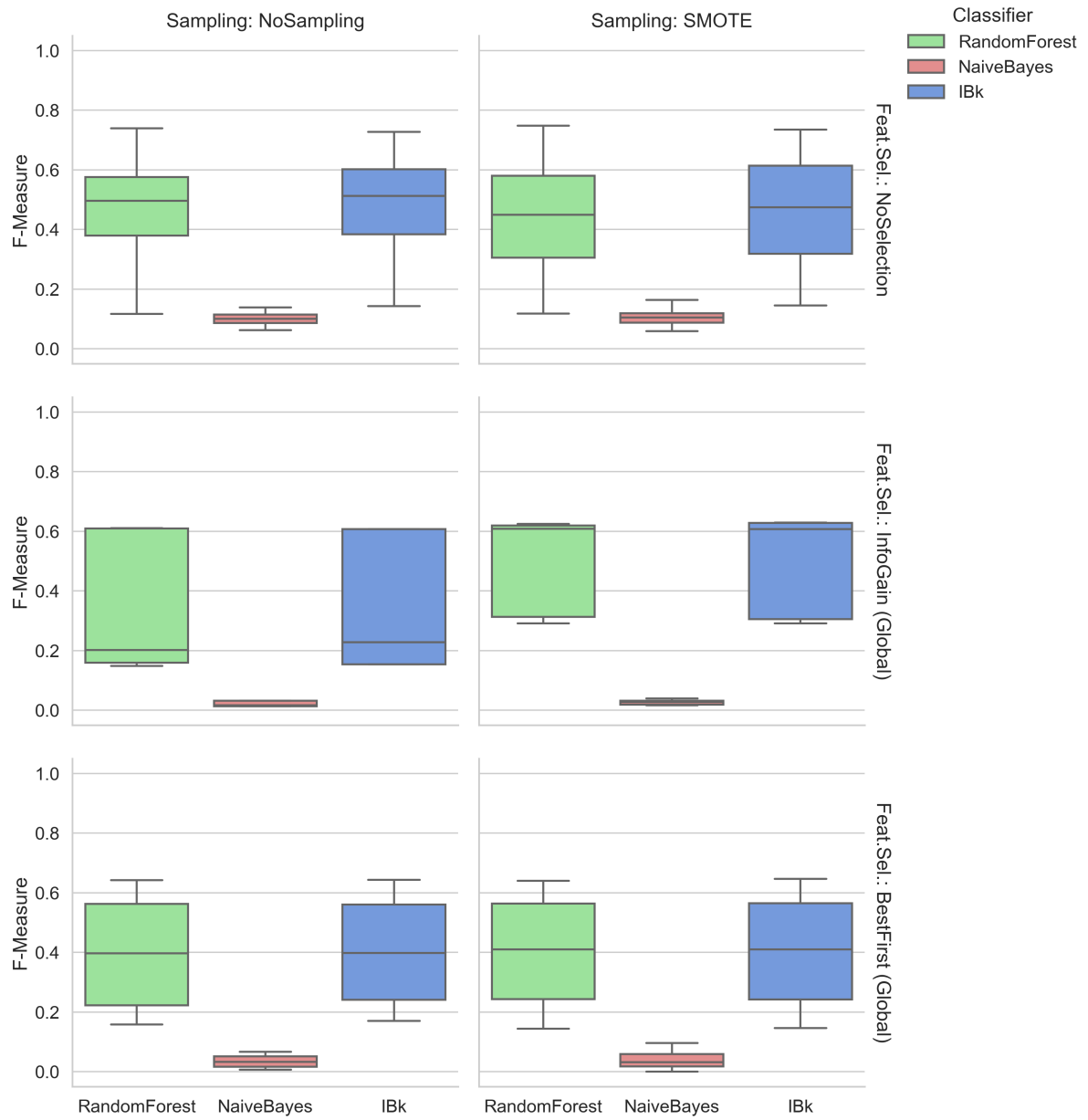


Figura 3: Distribuzione della Precision con feature Selection Globale - Apache Bookkeeper.

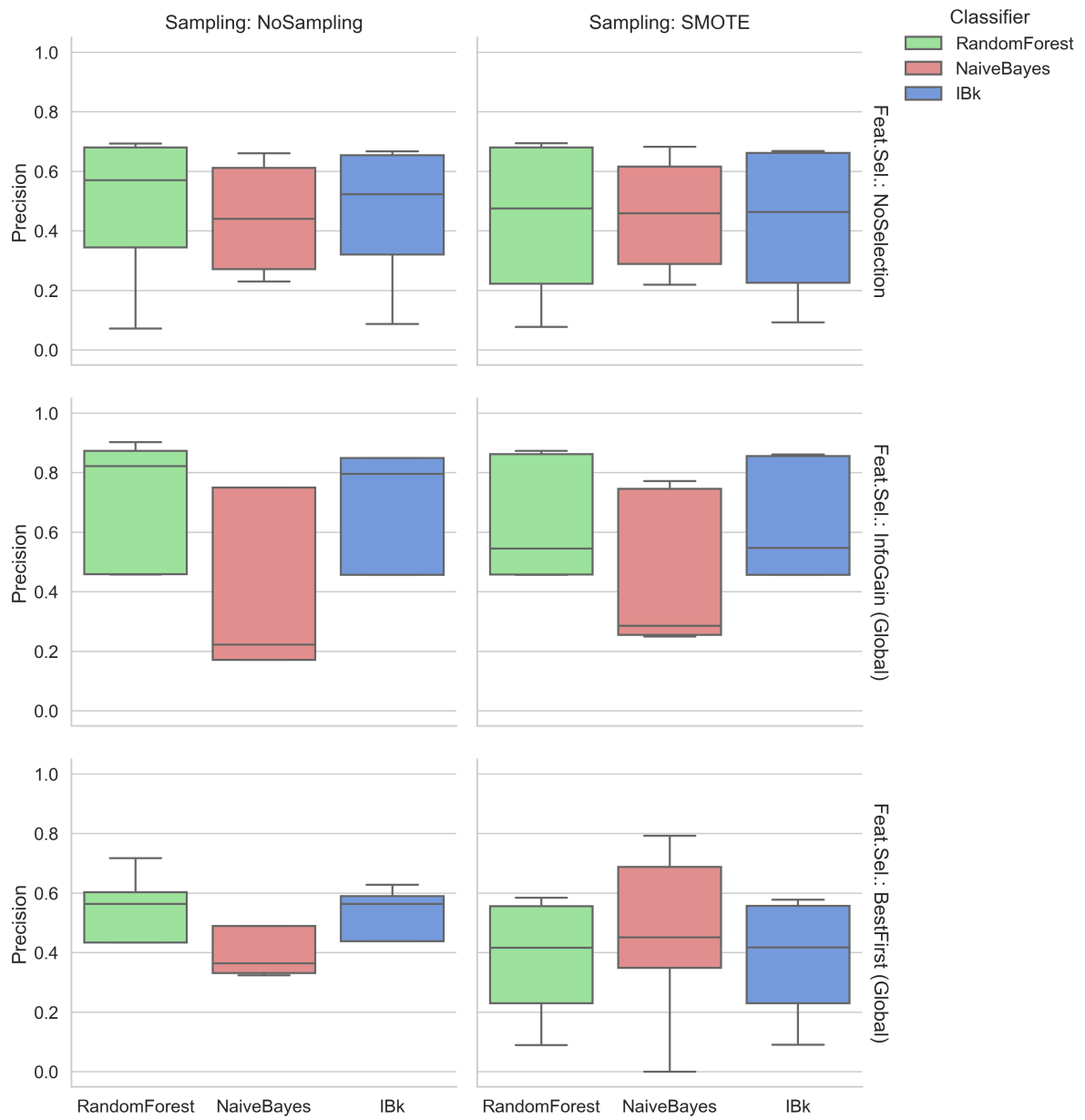


Figura 4: Distribuzione della Recall con feature Selection Globale - Apache Bookkeeper.

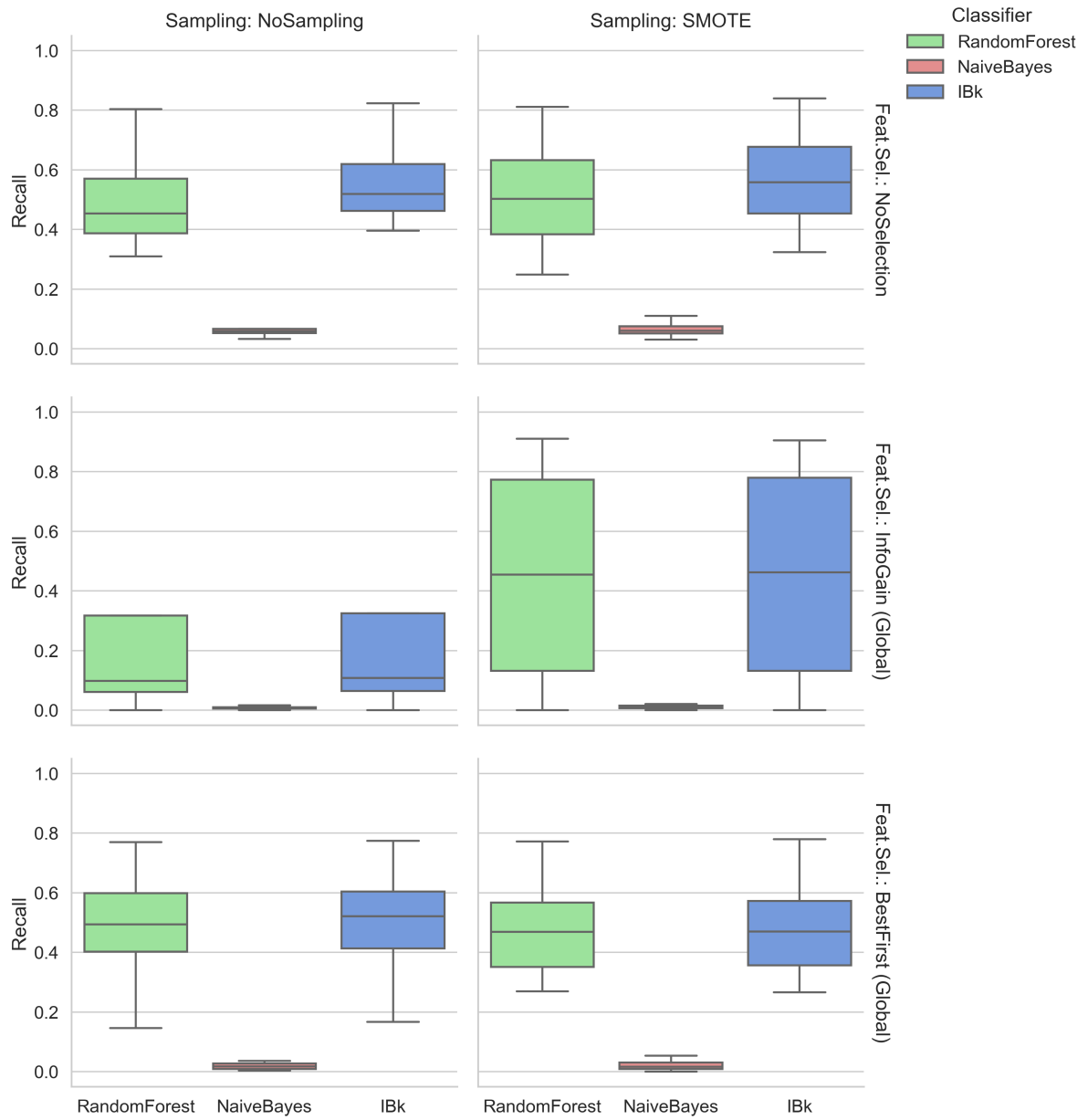


Figura 5: Distribuzione della statistica Kappa con feature Selection Globale - Apache Bookkeeper.

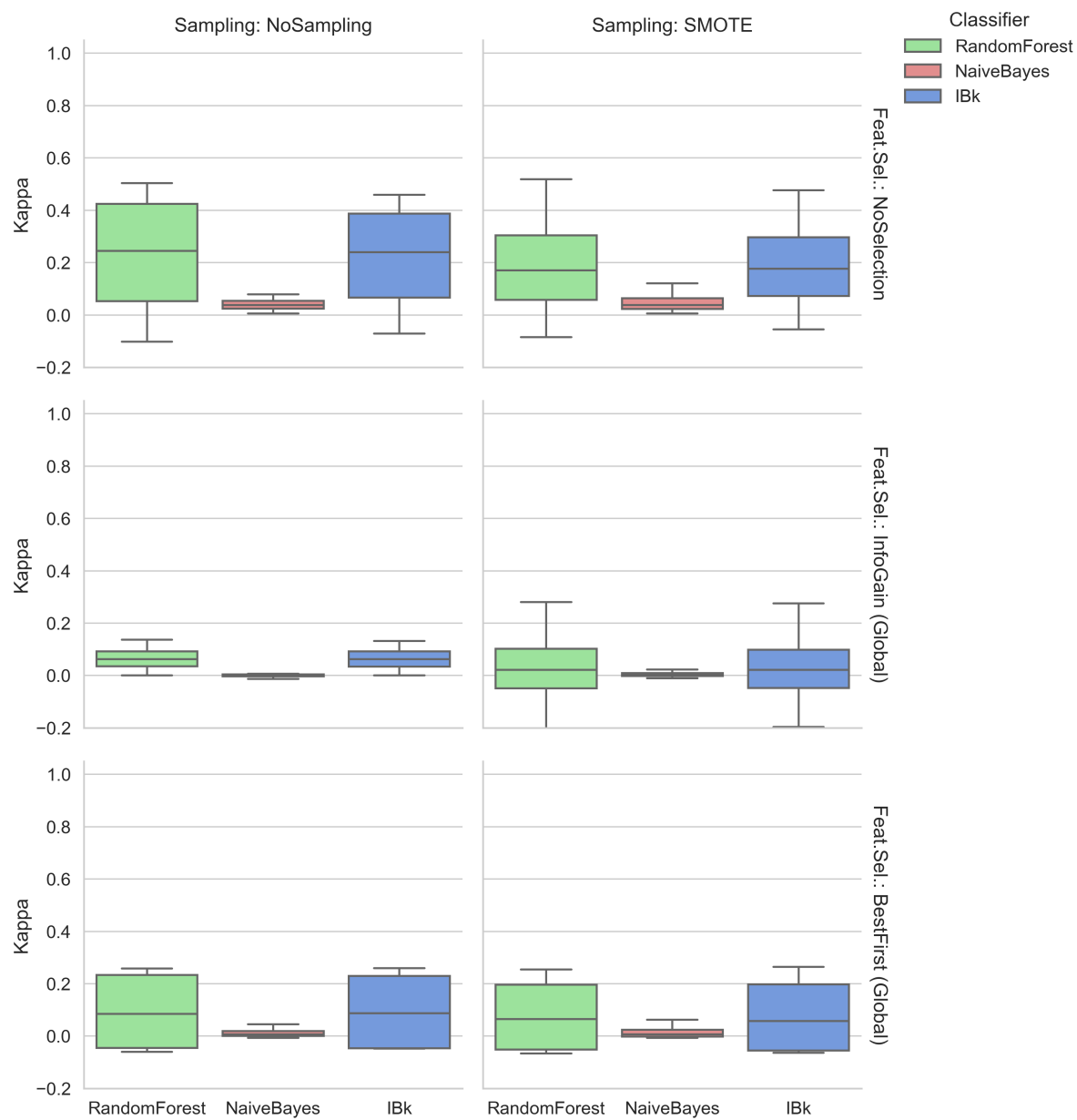


Figura 6: Distribuzione della metrica NPofB20 con feature Selection Globale - Apache Bookkeeper.

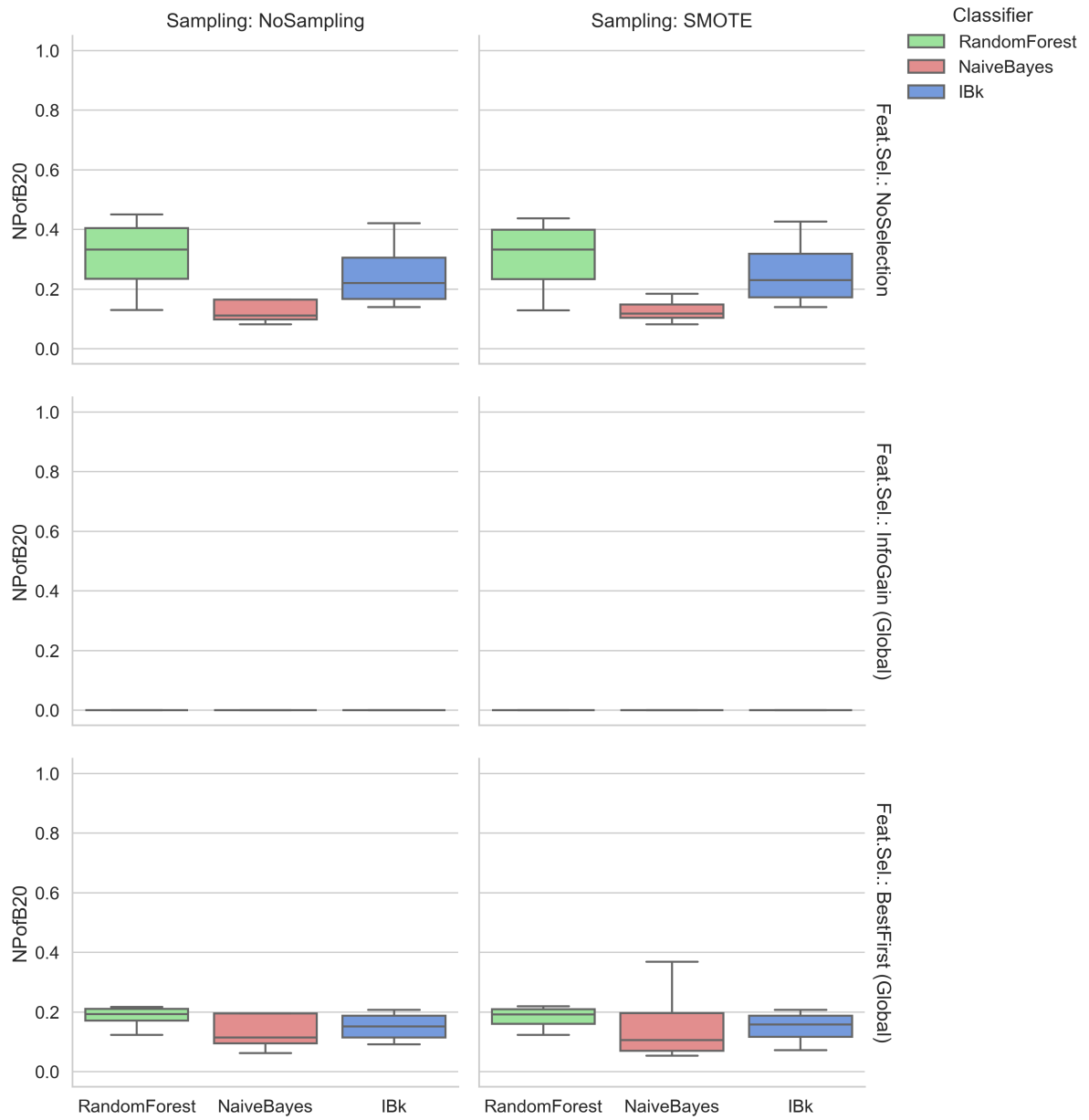


Figura 7: Distribuzione dell'AUC con Feature Selection Per-Fold - Apache Bookkeeper.

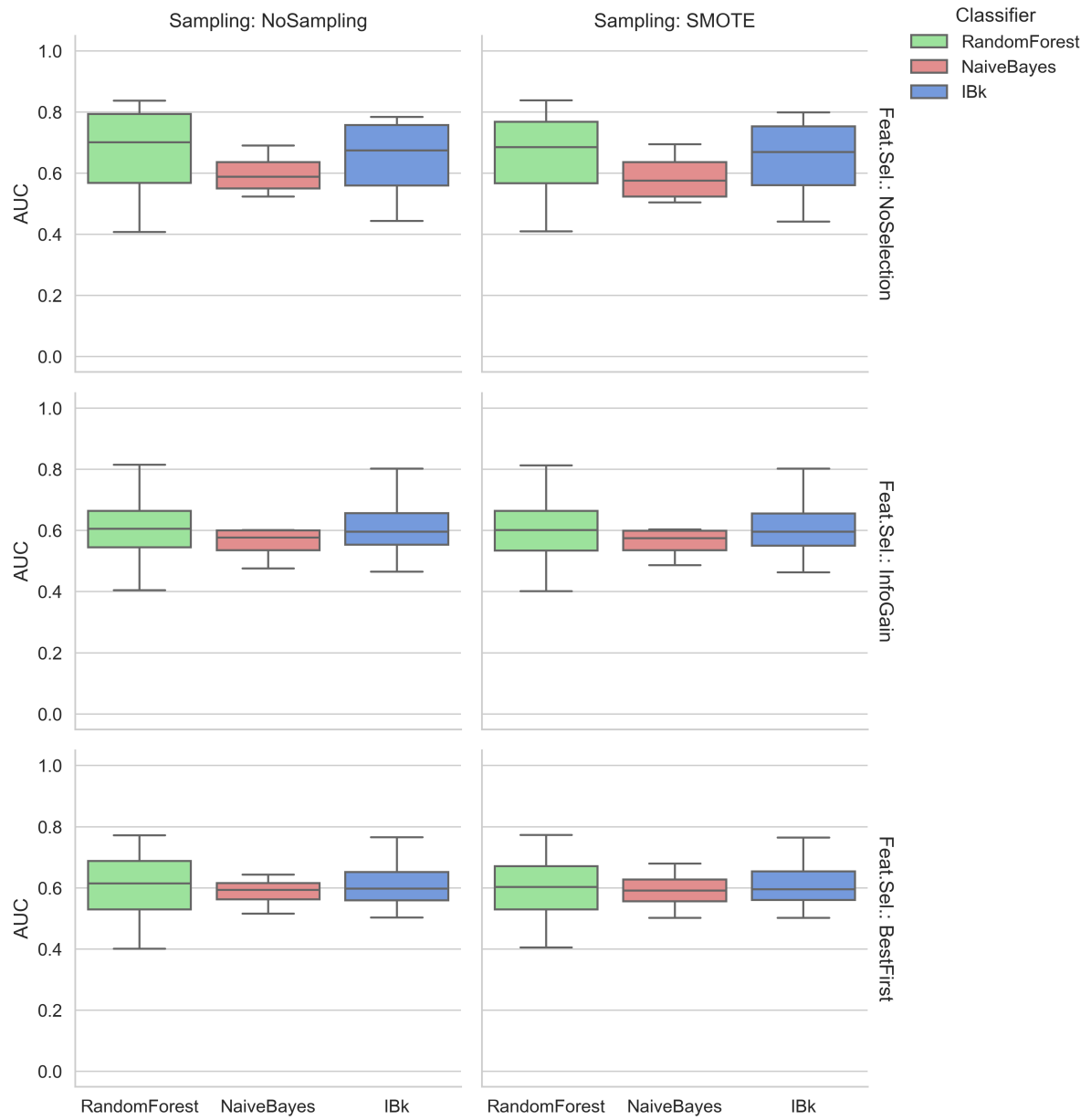


Figura 8: Distribuzione della F-Measure con Feature Selection Per-Fold - Apache Bookkeeper.

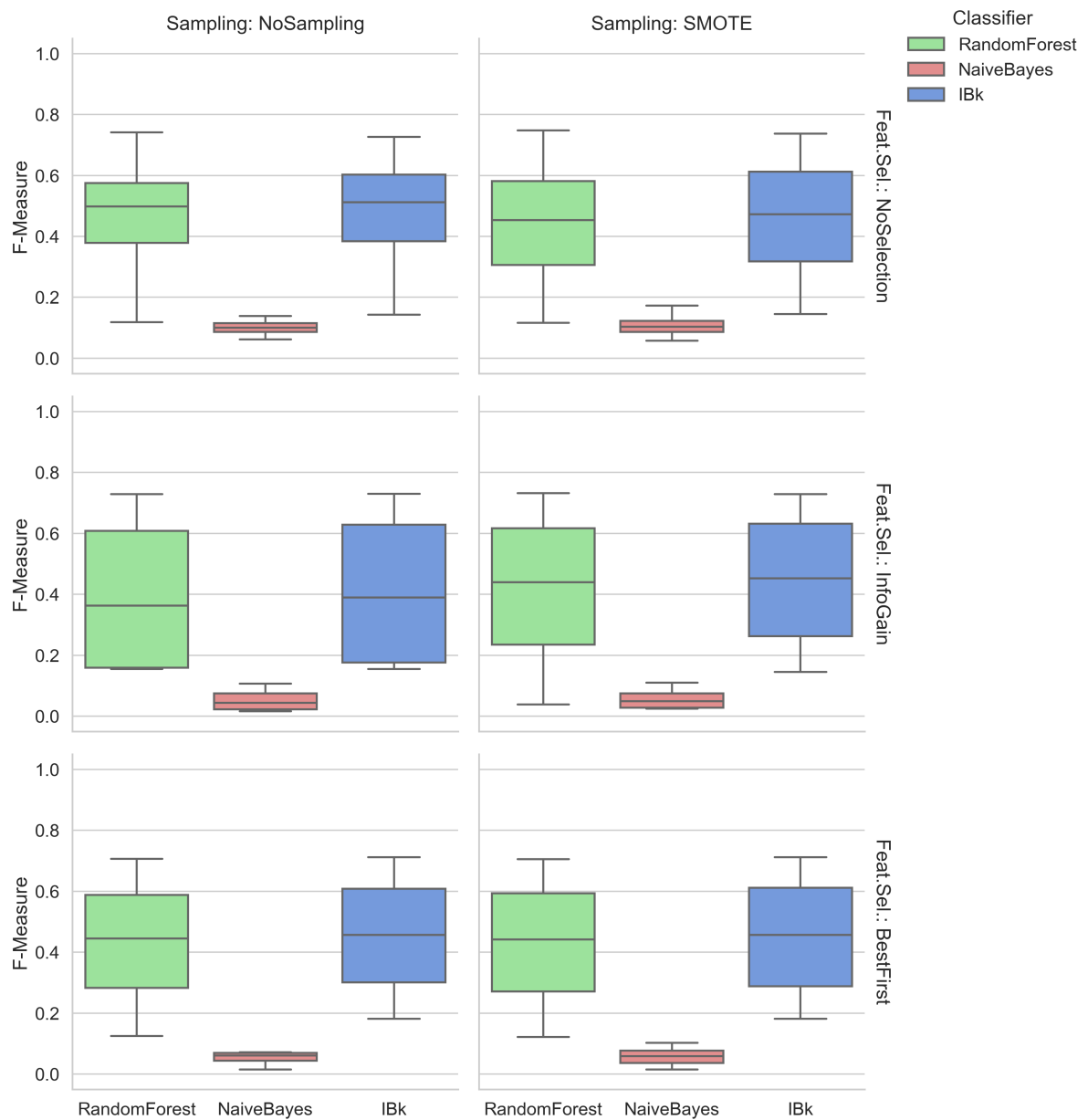


Figura 9: Distribuzione della Precision con Feature Selection Per-Fold - Apache Bookkeeper.

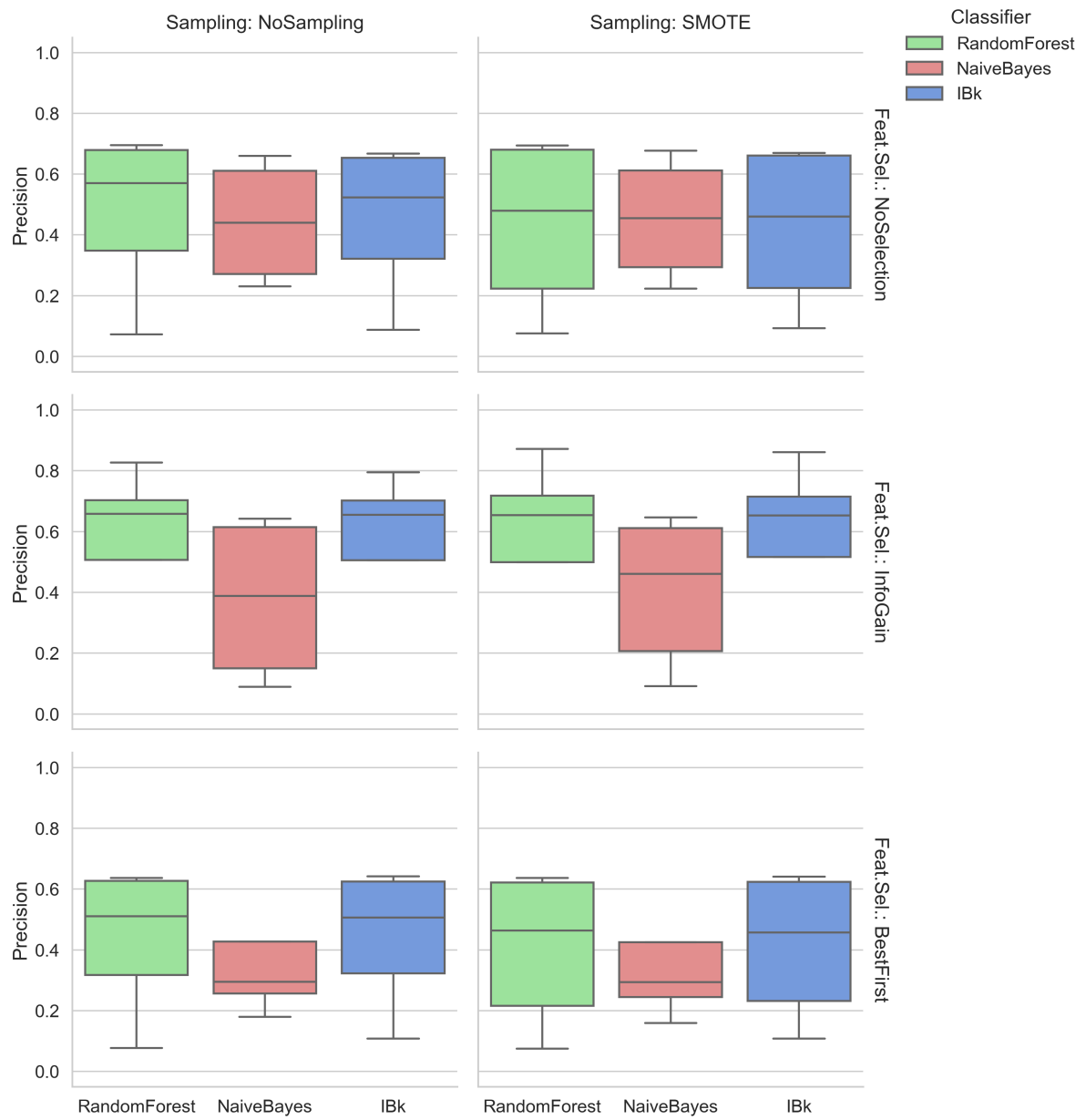


Figura 10: Distribuzione della Recall con Feature Selection Per-Fold - Apache Bookkeeper.

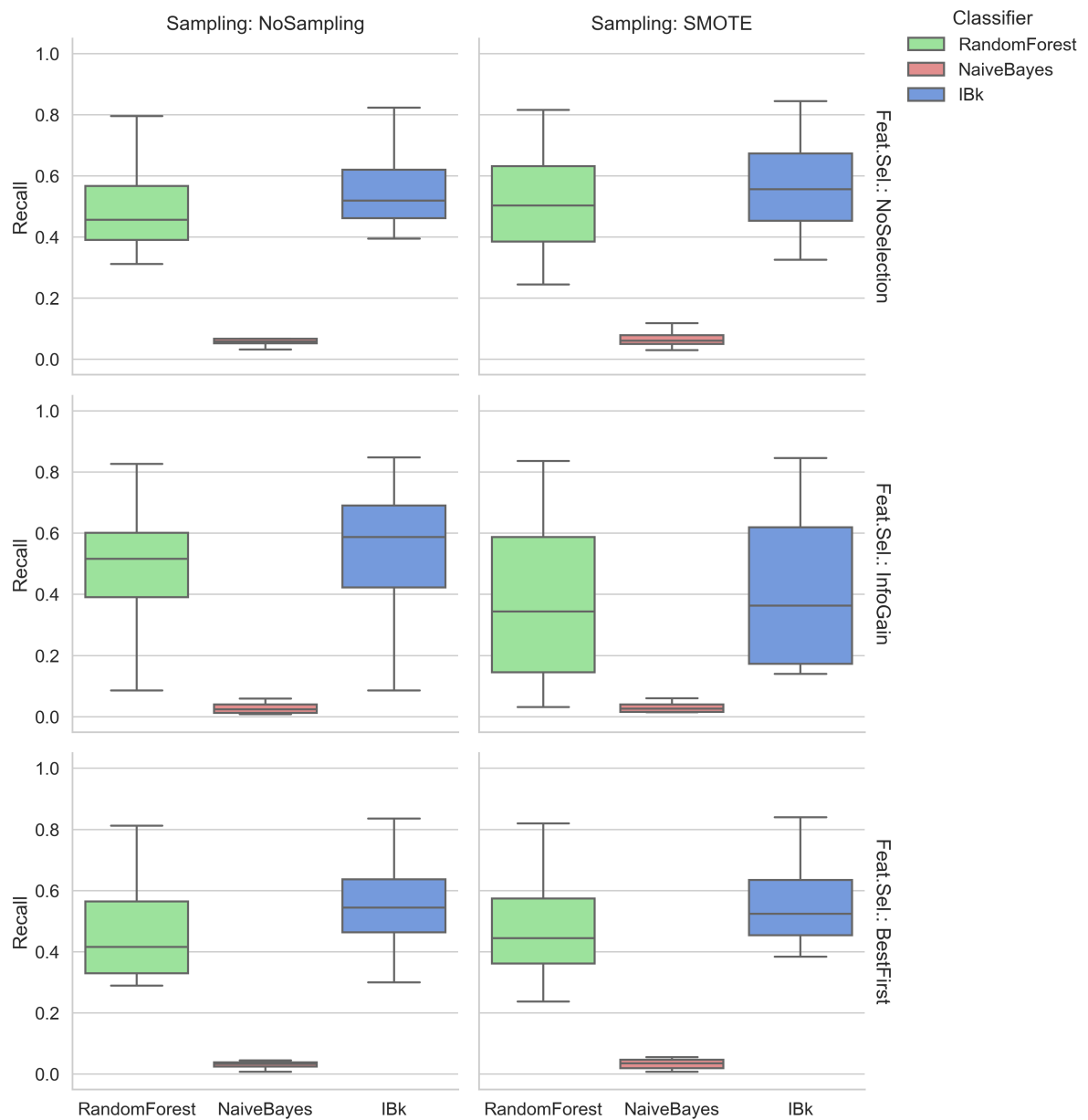


Figura 11: Distribuzione della statistica Kappa con Feature Selection Per-Fold - Apache Bookkeeper.

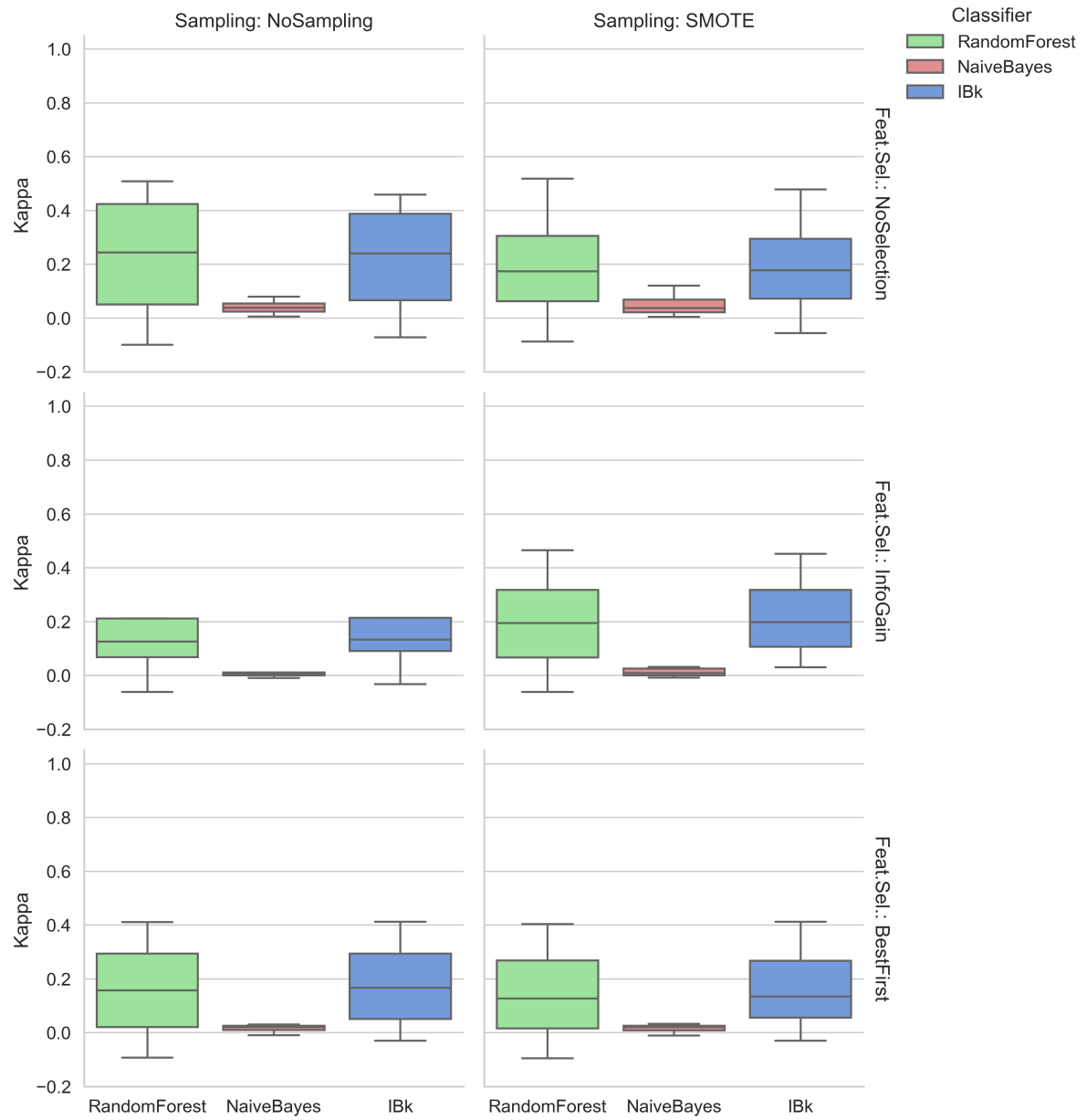


Figura 12: Distribuzione della metrica NPofB20 con Feature Selection Per-Fold - Apache Bookkeeper.

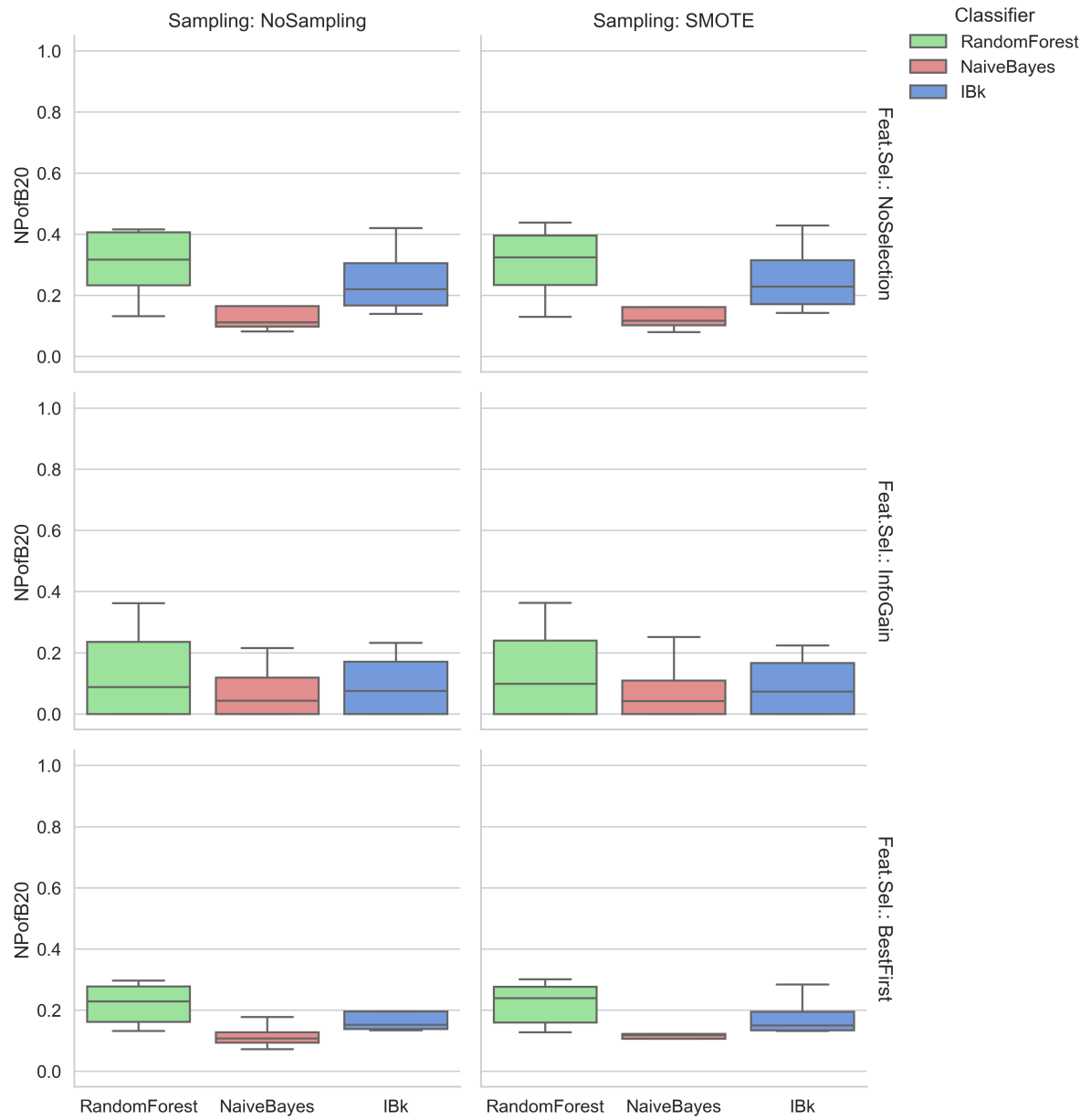


Tabella 2: Analisi di Correlazione Metriche vs Bugginess (Apache Bookkeeper)

Metrica	Pearson (r)	Spearman (ρ)
NR	0.1666	0.2816
NAuth	0.2638	0.2802
MAX_Churn	0.0313	0.2774
Churn	0.0367	0.2774
AVG_Churn	0.0283	0.2773
AVG_LOC_Added	0.0210	0.2729
MAX_LOC_Added	0.0237	0.2728
LOC_Added	0.0270	0.2728
MAX_LOC_Deleted	0.0671	0.2673
AVG_LOC_Deleted	0.0658	0.2671
LOC_Deleted	0.0778	0.2666
Global_LOC_Deleted	0.0568	0.1191
Global_MAX_LOC_Deleted	0.0453	0.1180
Global_AVG_LOC_Deleted	0.0483	0.1140
LOC	0.1147	0.1101
NumStatements	0.0839	0.0996
Cyclomatic	0.0999	0.0969
CognitiveComplexity	0.0908	0.0943
NestingDepth	0.0858	0.0912
CodeSmells	0.0874	0.0876
Global_NR	0.1011	0.0810
Global_NAuth	0.0874	0.0759
Global_Churn	0.0105	0.0674
Global_MAX_Churn	0.0048	0.0630
Global_AVG_Churn	-0.0082	0.0588
Global_LOC_Added	0.0027	0.0578
Global_MAX_LOC_Added	-0.0007	0.0549
Global_AVG_LOC_Added	-0.0159	0.0508
NumLocalVars	0.0572	0.0472
ReturnTypeComplexity	-0.0223	-0.0295
NumParams	-0.0178	-0.0227

Tabella 3: Pre-Refactoring Bookkeeper

Metrica	Valore
Info	
Release	4.2.1
Signature	main
Statiche	
LOC	144
Statements	73
Params	1
Vars	37
Cyclomatic	16
Cognitive	17
Nesting	3
Return	1
Smells	17
Locali	
NR	3
NAuth	1
Added	3
MAX_Add	1
AVG_Add	1
Deleted	3
MAX_Del	1
AVG_Del	1
Churn	6
MAX_Churn	2
AVG_Churn	2
Globali	
GLNR	27
GLNAuth	3
GLAdd	70
GLMAX_A	14
GLAVG_A	2.59
GLDel	26
GLMAX_D	3
GLAVG_D	0.96
GLChurn	96
GLMAX_C	16
GLAVG_C	3.56
Target Attuale	
Bugginess	TRUE

Tabella 4: Post-Refactoring (Decomposizione) Bookkeeper

Metrica	M1	M2	M3	M4	M5
<i>Sig.</i>	<i>rep</i>	<i>wait</i>	<i>main</i>	<i>set</i>	<i>bld</i>
Statiche					
LOC	21	23	39	12	20
Statements	8	9	25	1	18
Params	4	2	1	1	0
Vars	6	3	13	1	1
Cyclomatic	4	6	3	2	1
Cognitive	4	6	2	1	0
Nesting	2	2	1	1	0
Return	1	1	1	1	1
Smells	3	1	3	3	0
Locali					
NR	1	1	4	1	1
NAuth	1	1	1	1	1
Added	21	23	3	12	20
MAX_Add	21	23	1	12	20
AVG_Add	21	23	0.75	12	20
Deleted	0	0	108	0	0
MAX_Del	0	0	105	0	0
AVG_Del	0	0	27	0	0
Churn	21	23	111	12	20
MAX_Ch	21	23	105	12	20
AVG_Ch	21	23	27.75	12	20
Globali					
GLNR	1	1	28	1	1
GLNAuth	1	1	3	1	1
GLAdd	21	23	70	12	20
GLMAX_A	21	23	14	12	20
GLAVG_A	21	23	2.5	12	20
GLDel	0	0	131	0	0
GLMAX_D	0	0	105	0	0
GLAVG_D	0	0	4.67	0	0
GLChurn	21	23	201	12	20
GLMAX_C	21	23	105	12	20
GLAVG_C	21	23	7.18	12	20
Predizione					
Prob.	31%	21%	20%	40%	40%
Risk	LO	LO	LO	LO	LO

Legenda: M1:report, M2:waitFor, M3:main, M4:setup, M5:build

Tabella 5: What-If analysis Bookkeeper no code smell

Dataset	Actual Bugs	Predicted Bugs (E)
Dataset A	6790	3835
Dataset B+	2755	2129
Dataset B (Fixed)	-	1951
Dataset C	4035	1706

Tabella 6: Risultati Aggregati per Bookkeeper

Classifier	Sampling	Feat. Sel.	Prec.	Rec.	F-Meas.	AUC	Kappa	NPofB20
IBk	NoSampling	BestFirst	0.4408	0.5561	0.4518	0.6154	0.1793	0.1819
IBk	NoSampling	InfoGain	0.5534	0.5265	0.4156	0.6148	0.1715	0.0952
IBk	NoSampling	NoSelection	0.4506	0.5636	0.4732	0.6436	0.2157	0.2508
IBk	SMOTE	BestFirst	0.4118	0.5748	0.4495	0.6145	0.1664	0.1791
IBk	SMOTE	InfoGain	0.5796	0.4282	0.4441	0.6140	0.2207	0.0923
IBk	SMOTE	NoSelection	0.4219	0.5688	0.4564	0.6440	0.1923	0.2565
NaiveBayes	NoSampling	BestFirst	0.3888	0.0295	0.0523	0.5858	0.0160	0.1154
NaiveBayes	NoSampling	InfoGain	0.3769	0.0287	0.0527	0.5578	0.0074	0.0755
NaiveBayes	NoSampling	NoSelection	0.4427	0.0604	0.1002	0.5975	0.0402	0.1523
NaiveBayes	SMOTE	BestFirst	0.3803	0.0325	0.0576	0.5917	0.0158	0.1362
NaiveBayes	SMOTE	InfoGain	0.4005	0.0310	0.0570	0.5599	0.0108	0.0742
NaiveBayes	SMOTE	NoSelection	0.4506	0.0677	0.1095	0.5876	0.0479	0.1622
RandomForest	NoSampling	BestFirst	0.4337	0.4829	0.4294	0.6004	0.1575	0.2183
RandomForest	NoSampling	InfoGain	0.5586	0.4827	0.4025	0.6065	0.1608	0.1290
RandomForest	NoSampling	NoSelection	0.4717	0.5037	0.4605	0.6612	0.2222	0.3018
RandomForest	SMOTE	BestFirst	0.4050	0.5040	0.4279	0.5961	0.1455	0.2198
RandomForest	SMOTE	InfoGain	0.5592	0.3859	0.4117	0.6033	0.1954	0.1342
RandomForest	SMOTE	NoSelection	0.4300	0.5144	0.4397	0.6528	0.1917	0.3008

A.3 Apache OpenJPA

Figura 13: Distribuzione dell'AUC con Feature Selection Globale - Apache OpenJPA.

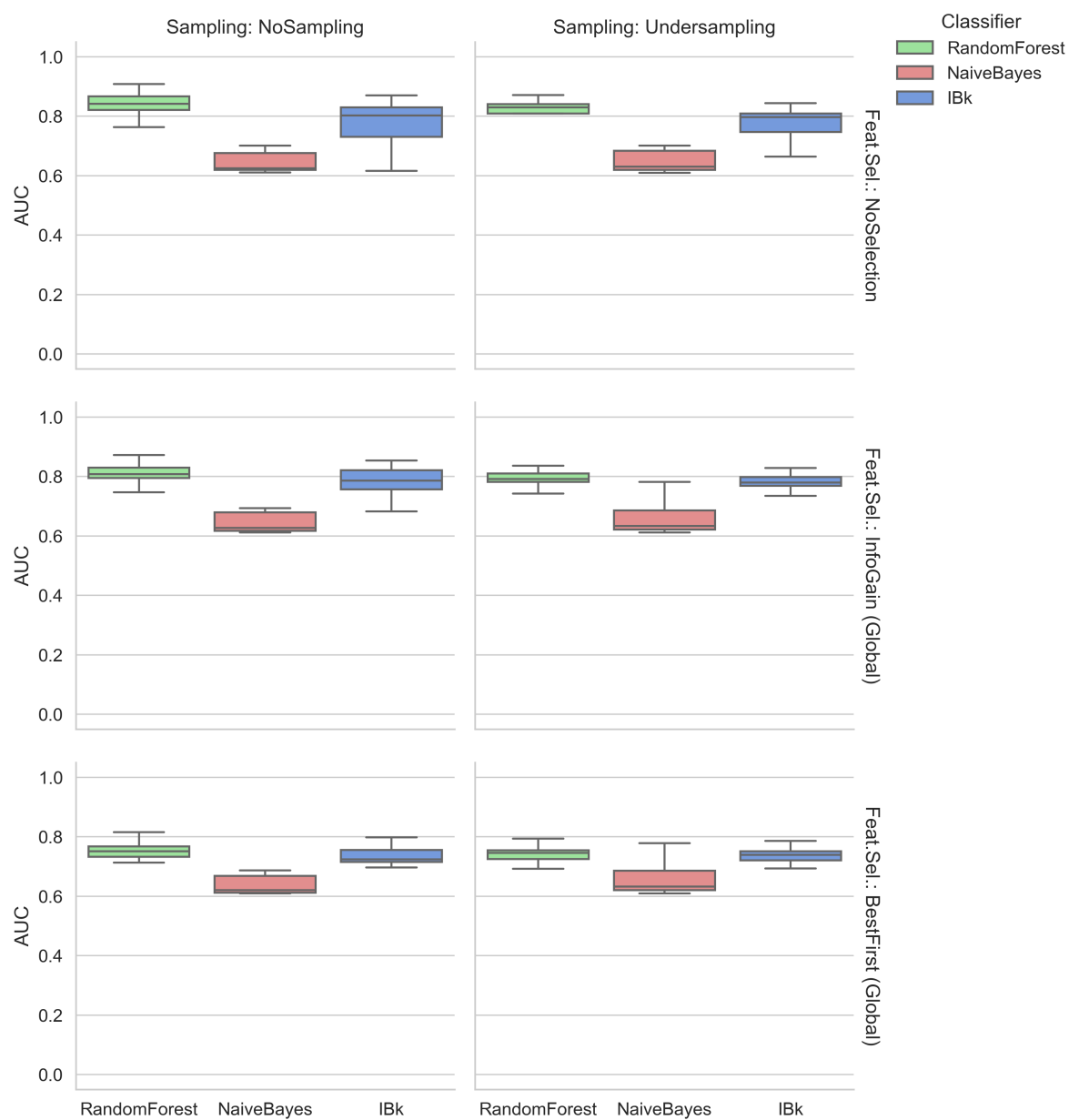


Figura 14: Distribuzione della F-Measure con Feature Selection Globale - Apache OpenJPA.

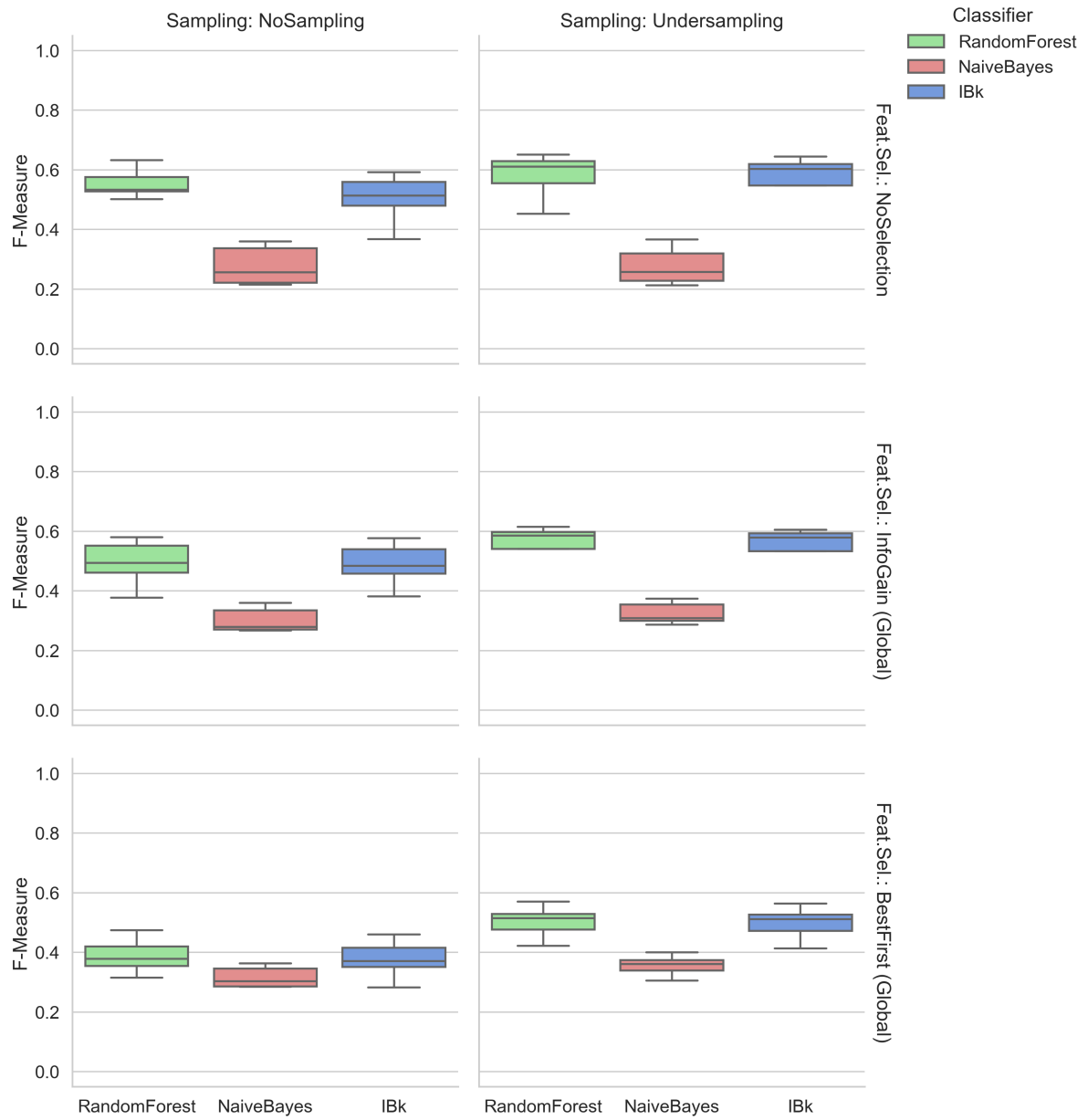


Figura 15: Distribuzione della Precision con Feature Selection Globale - Apache OpenJPA.

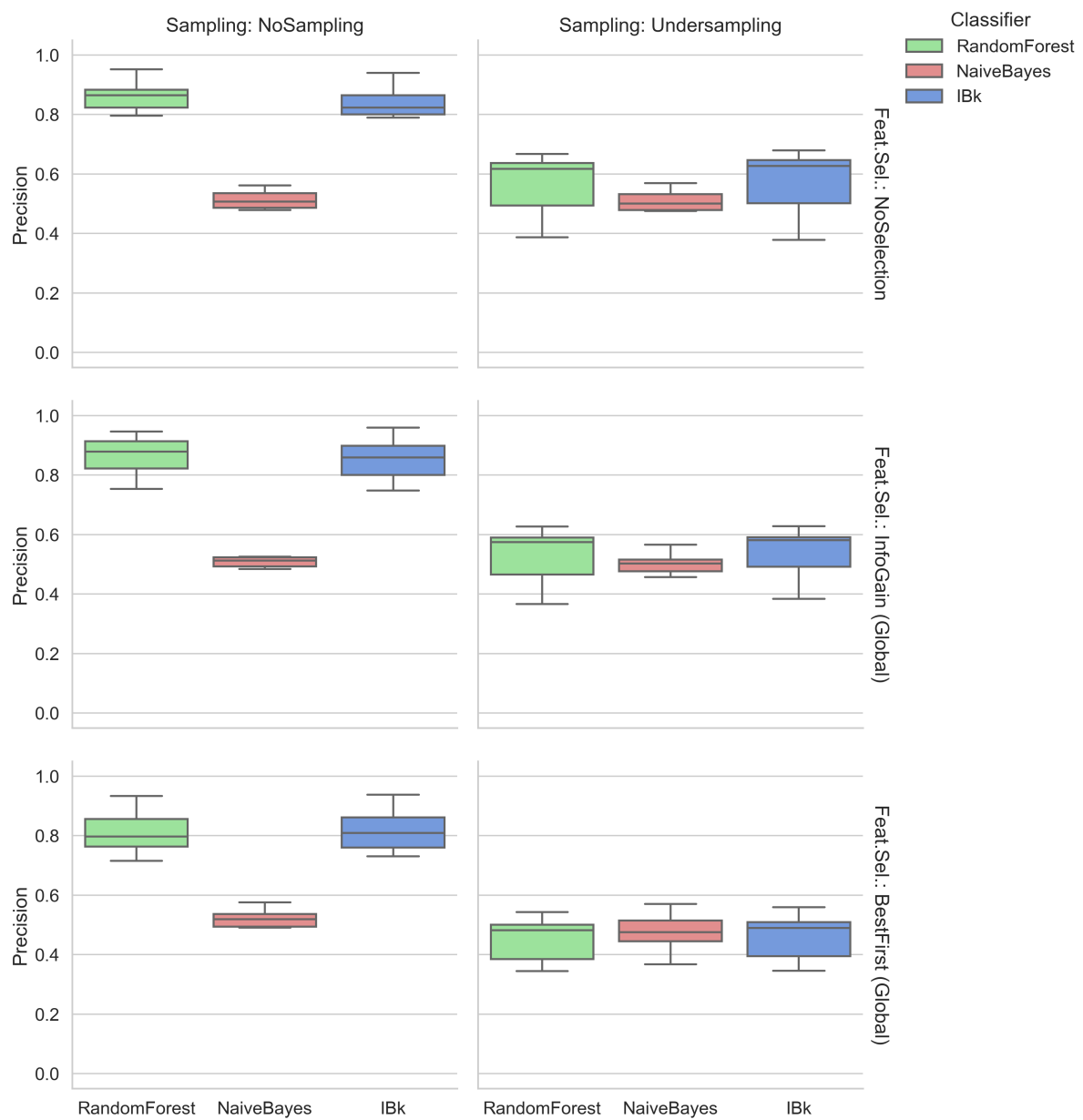


Figura 16: Distribuzione della Recall con Feature Selection Globale - Apache OpenJPA.

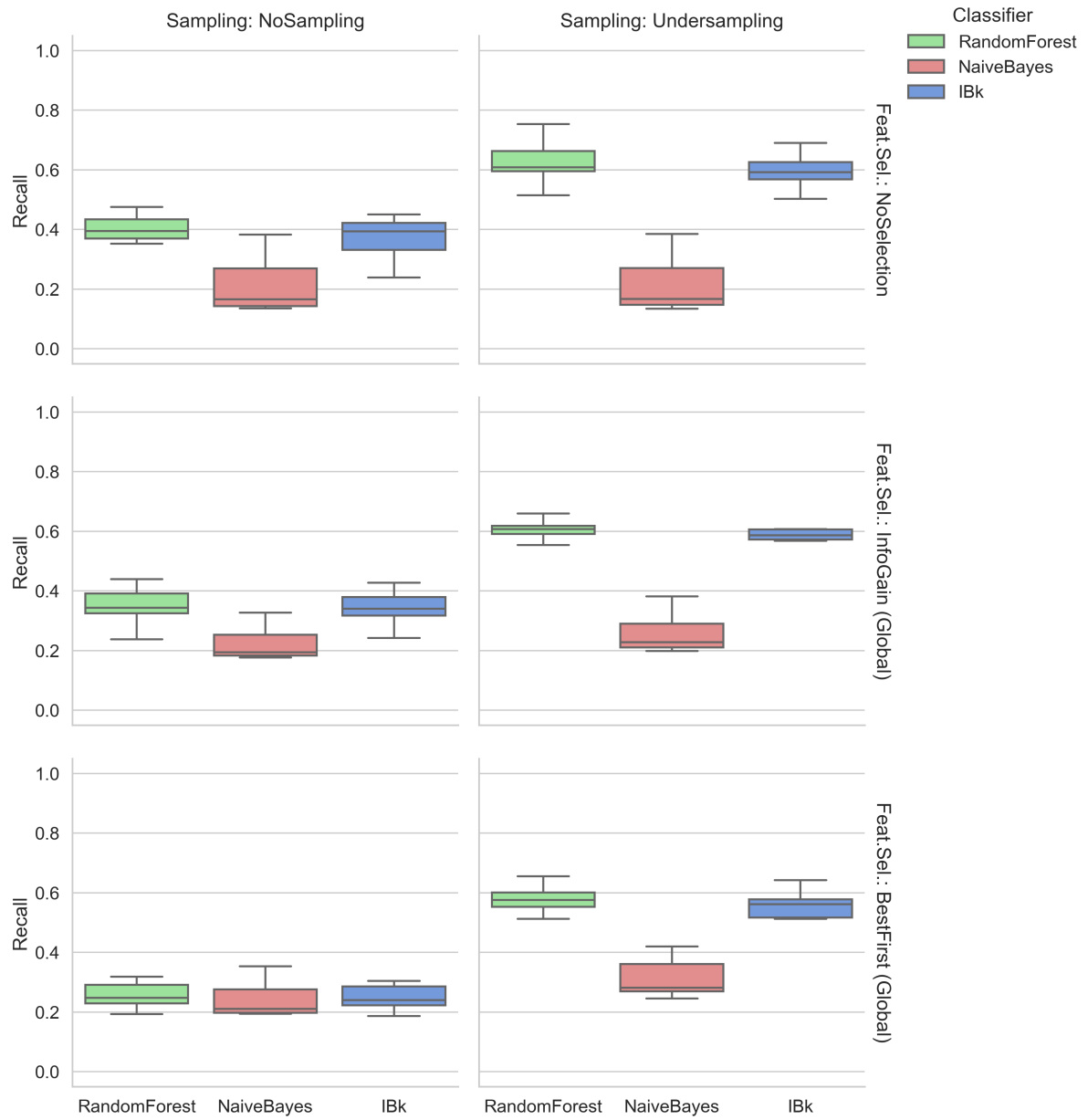


Figura 17: Distribuzione della statistica Kappa con Feature Selection Globale - Apache OpenJPA.

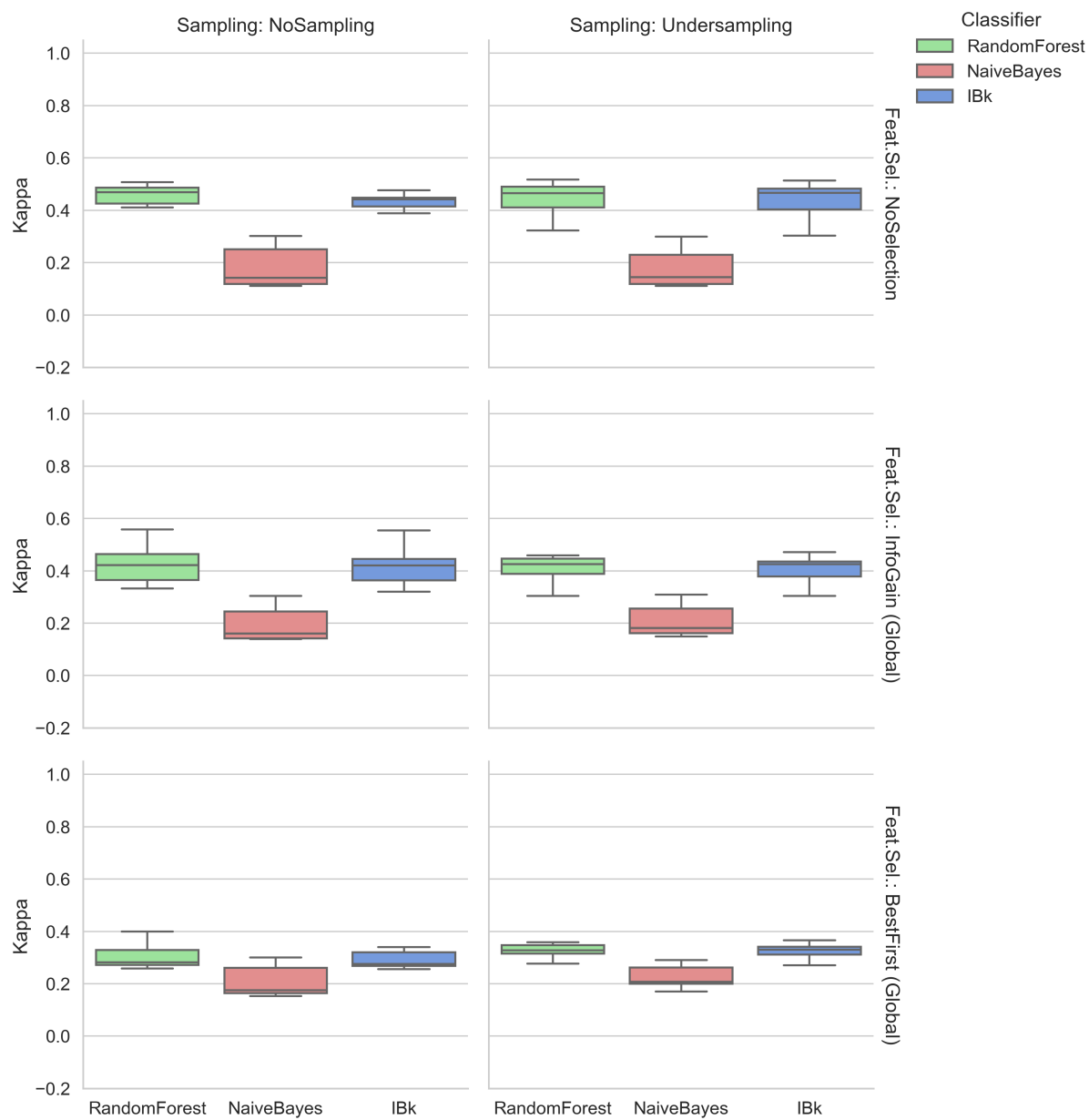


Figura 18: Distribuzione della metrica NPofB20 con Feature Selection Globale - Apache OpenJPA.

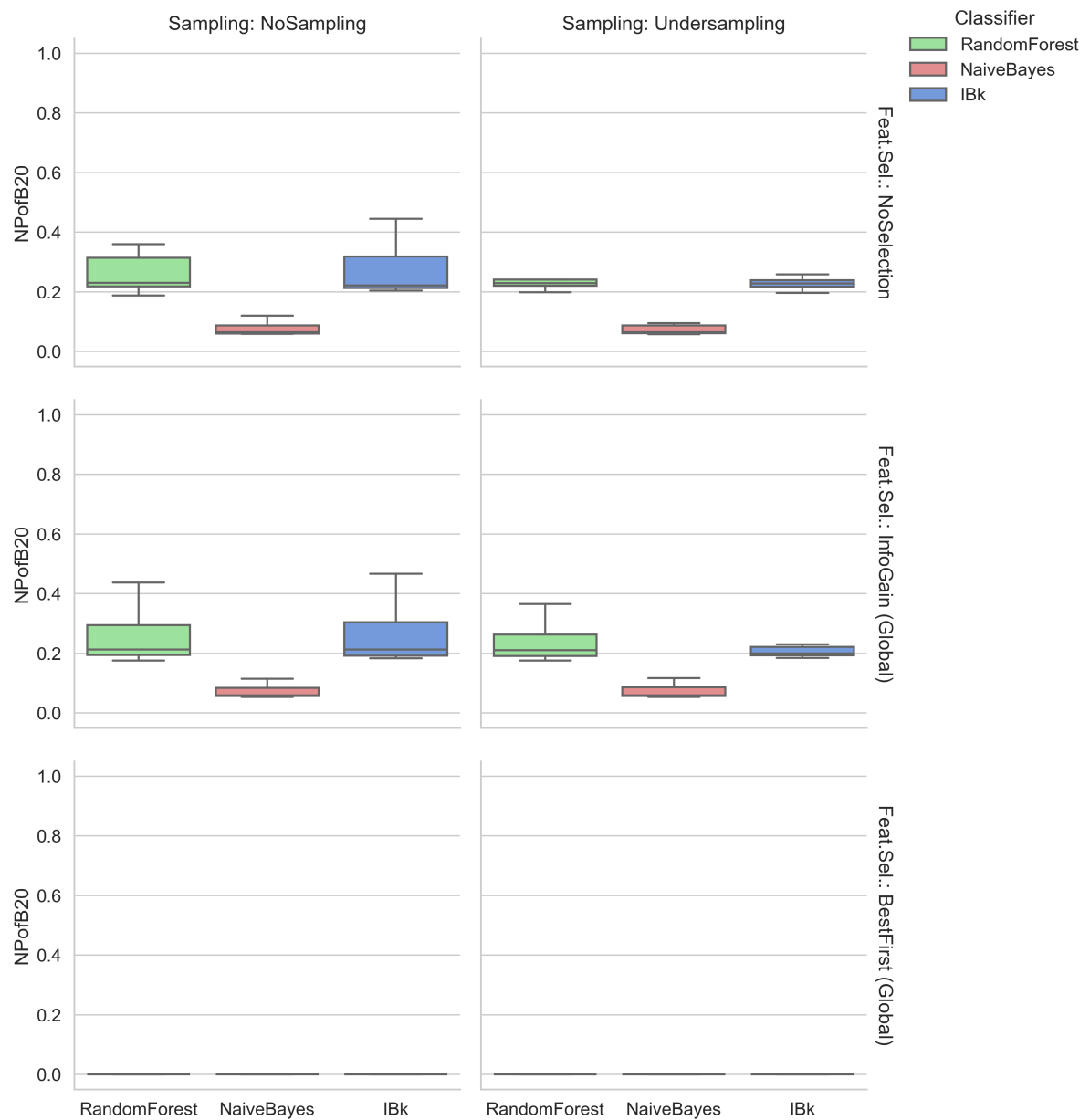


Figura 19: Distribuzione dell'AUC con Feature Selection Per-Fold - Apache OpenJPA.

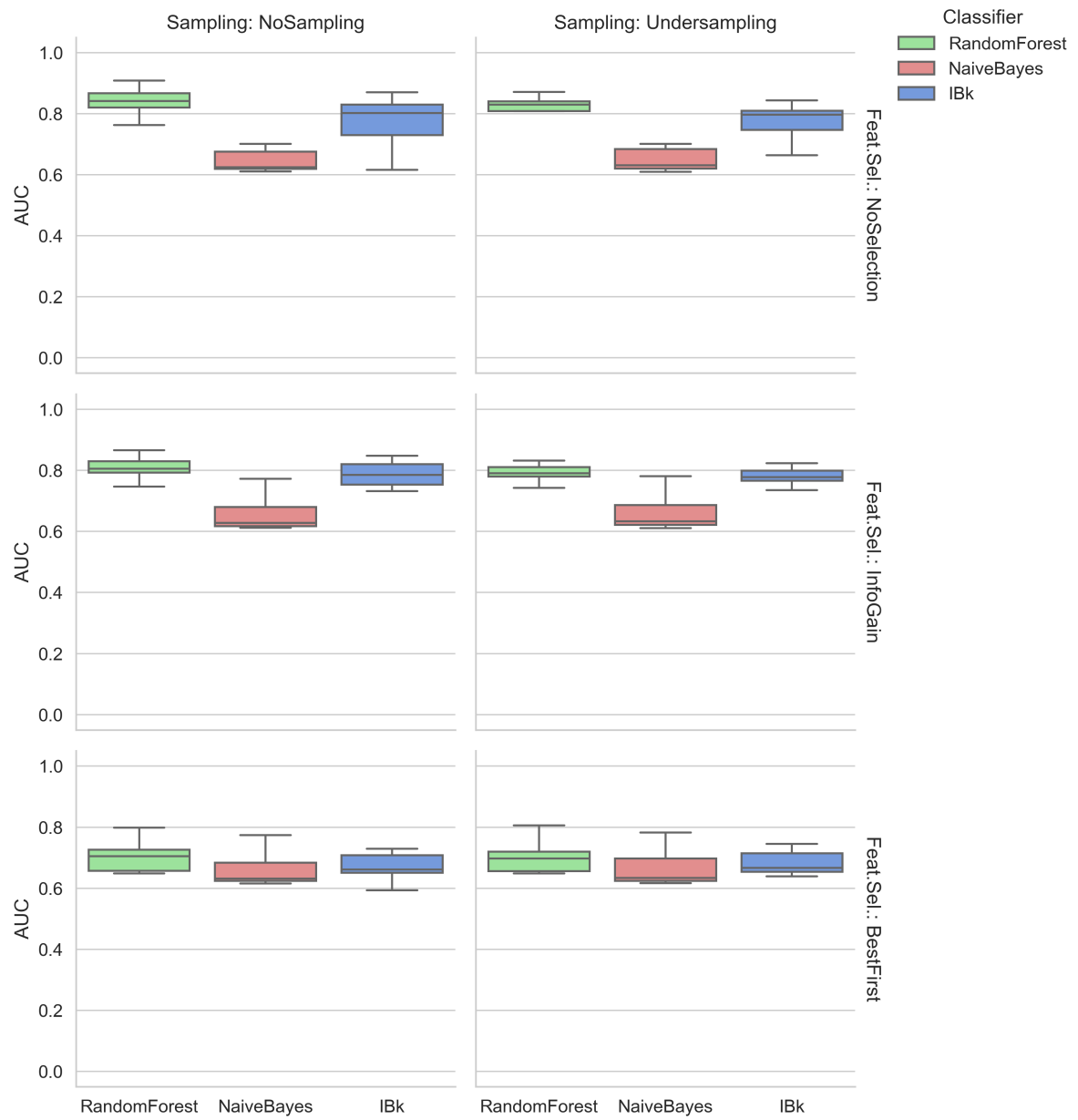


Figura 20: Distribuzione della F-Measure con Feature Selection Per-Fold - Apache OpenJPA.

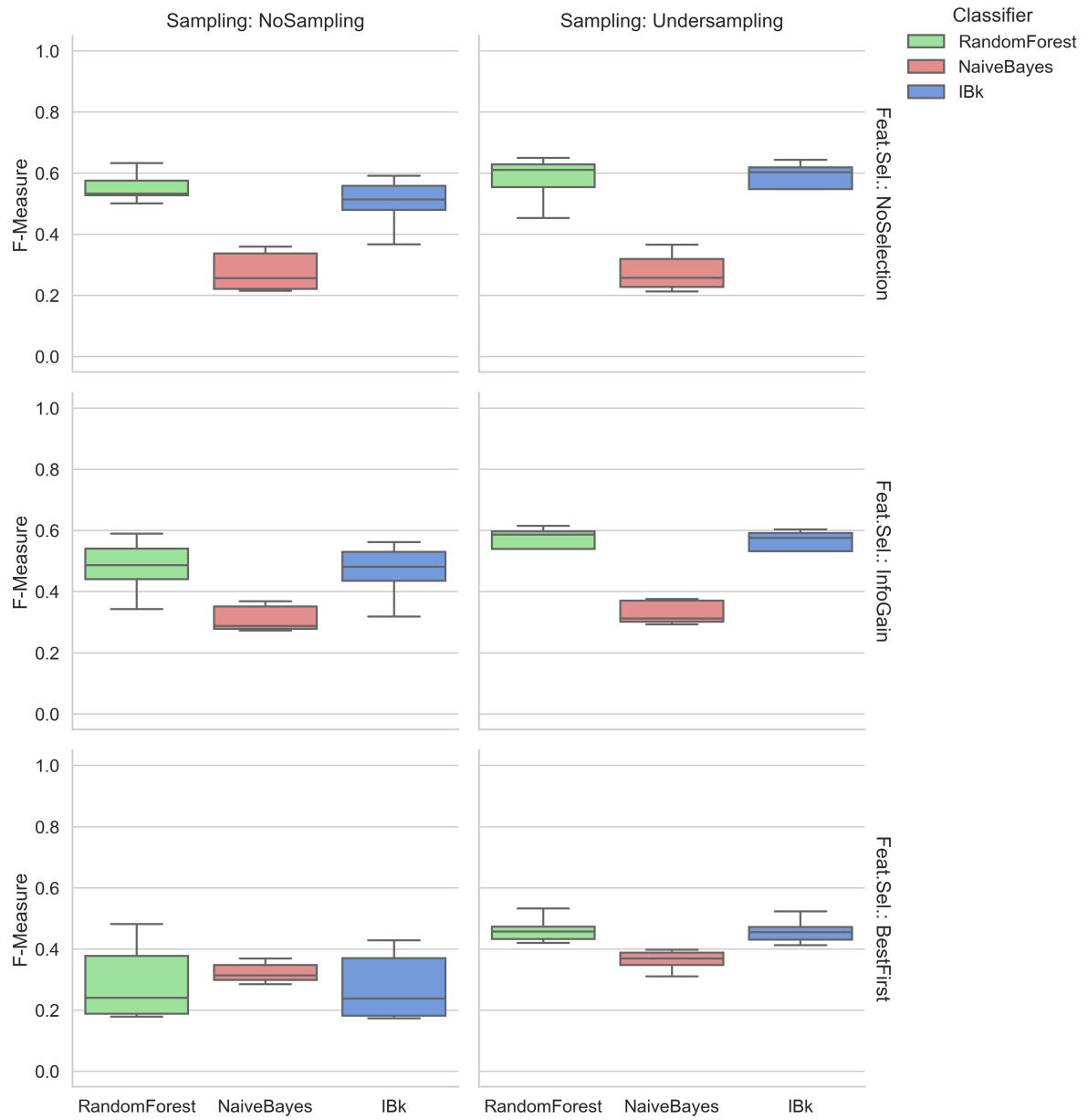


Figura 21: Distribuzione della Precision con Feature Selection Per-Fold - Apache OpenJPA.

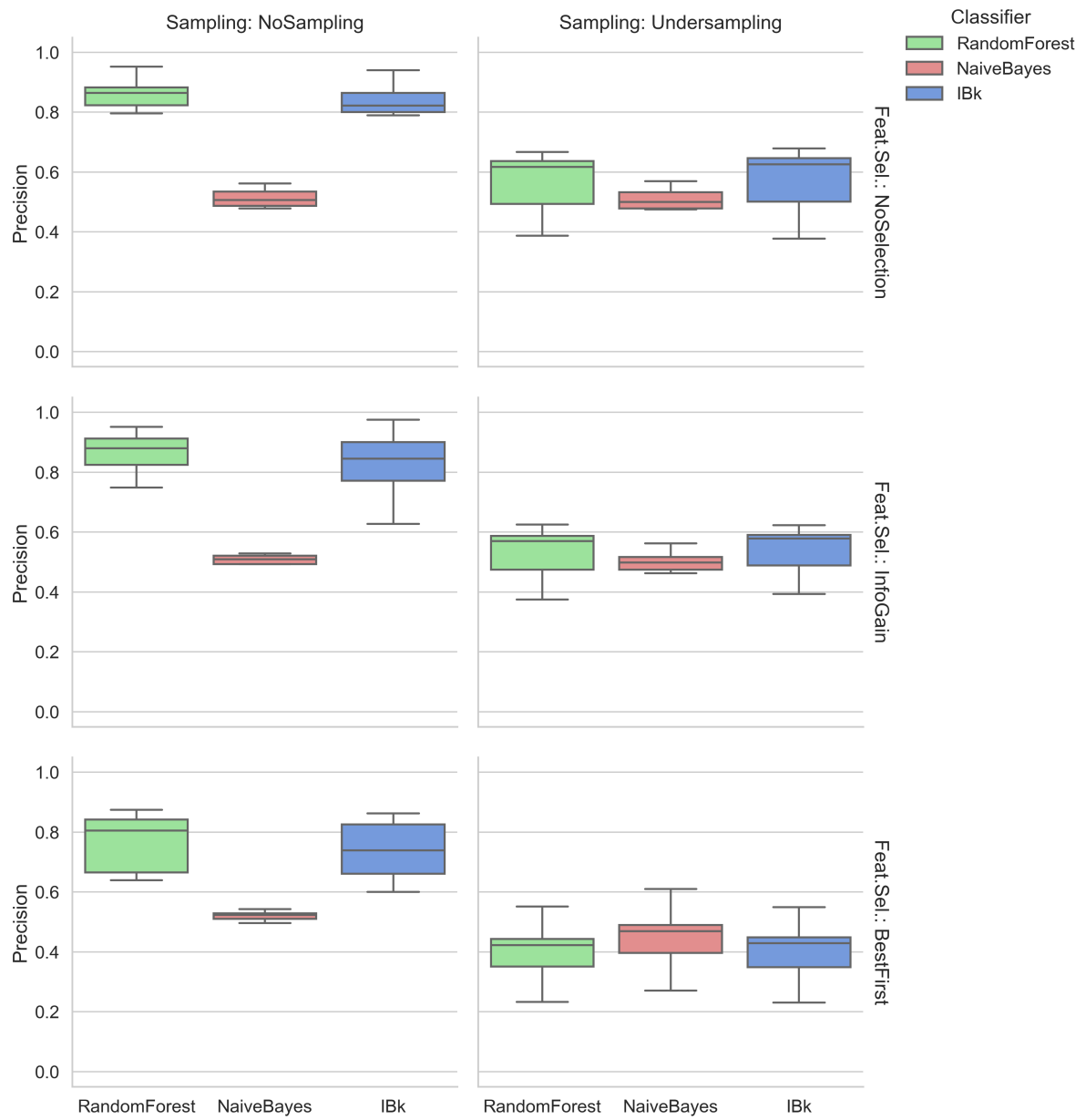


Figura 22: Distribuzione della Recall con Feature Selection Per-Fold - Apache OpenJPA.

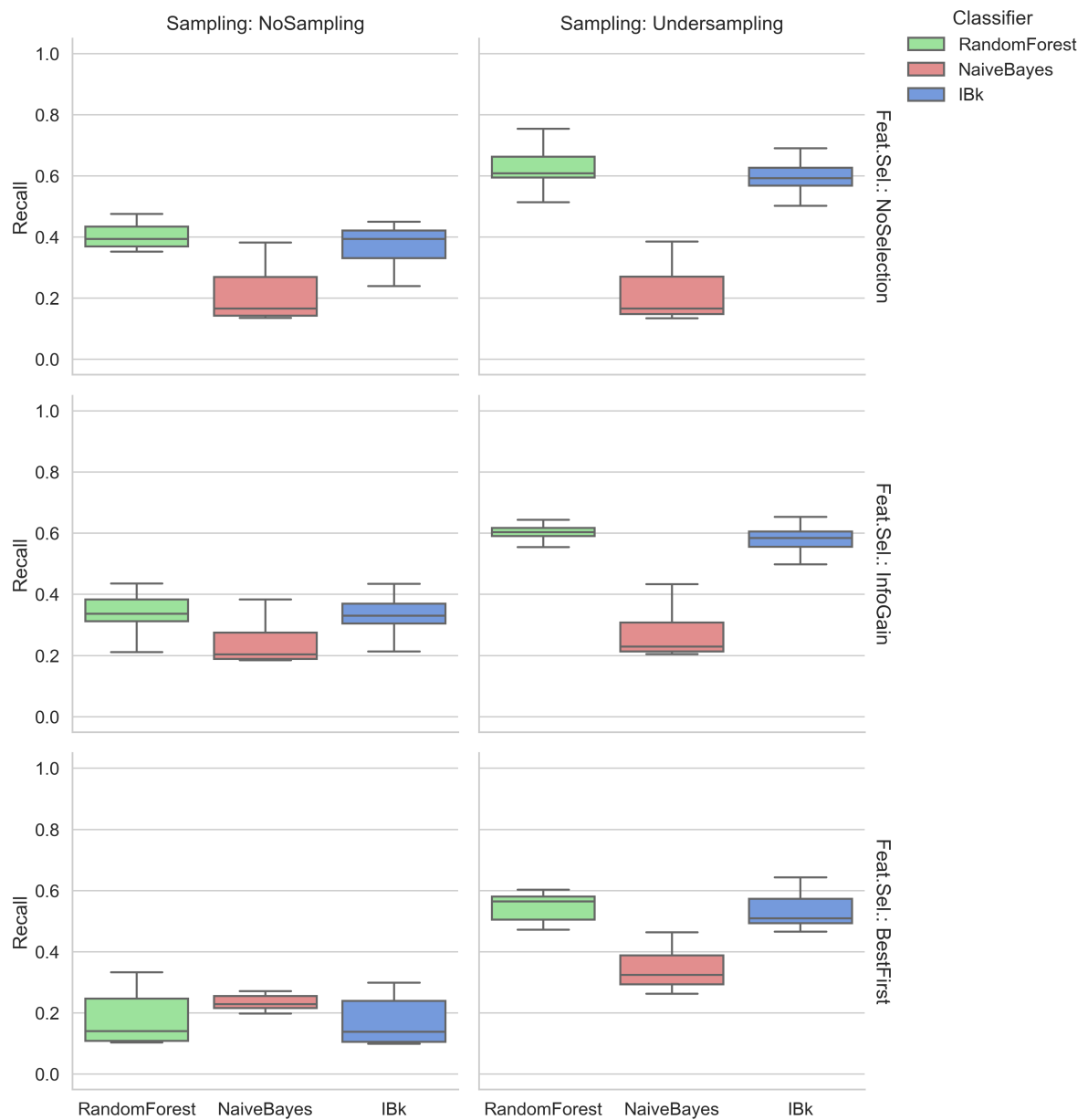


Figura 23: Distribuzione della statistica Kappa con Feature Selection Per-Fold - Apache OpenJPA.

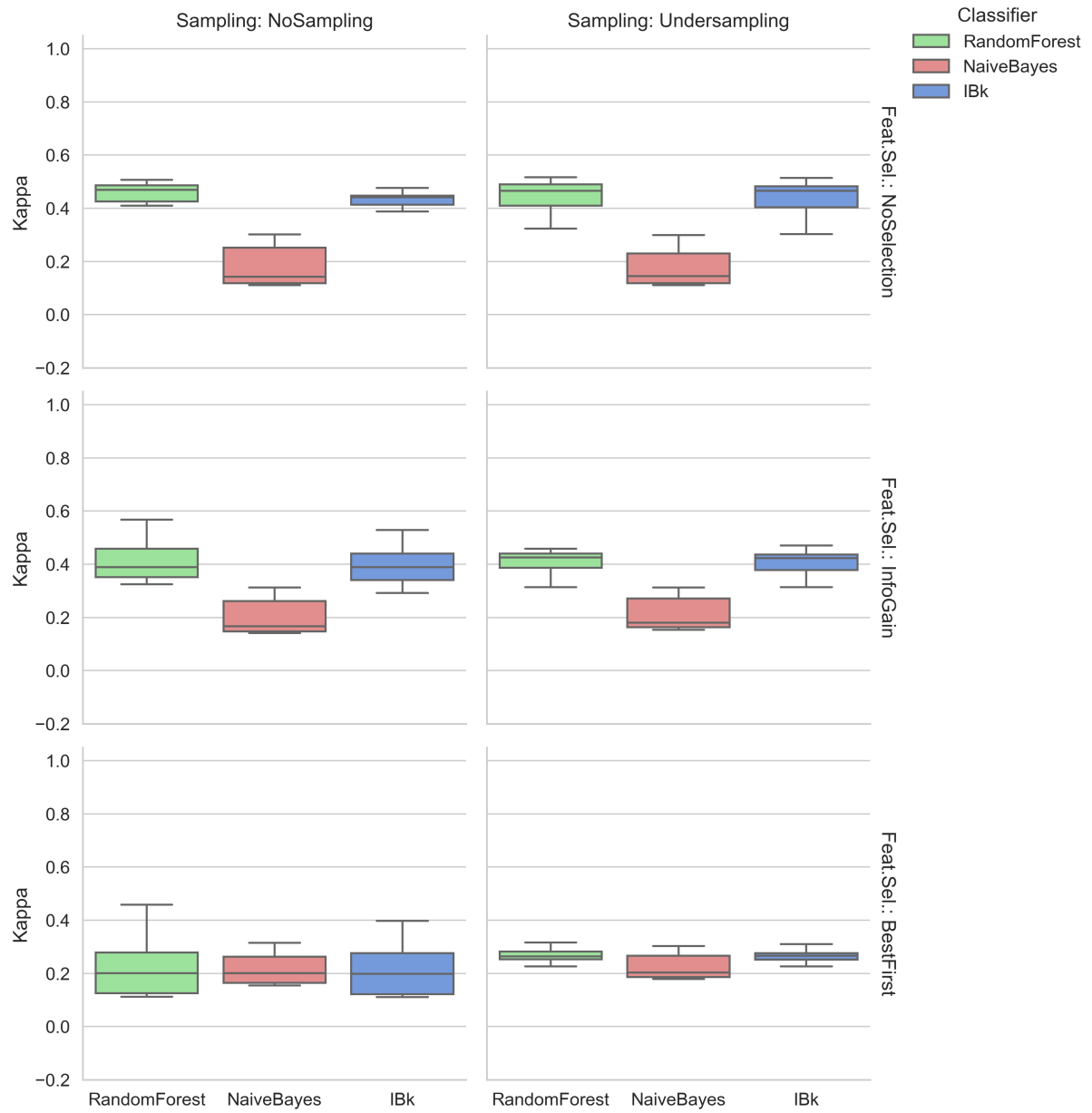


Figura 24: Distribuzione della metrica NPofB20 con Feature Selection Per-Fold - Apache OpenJPA.

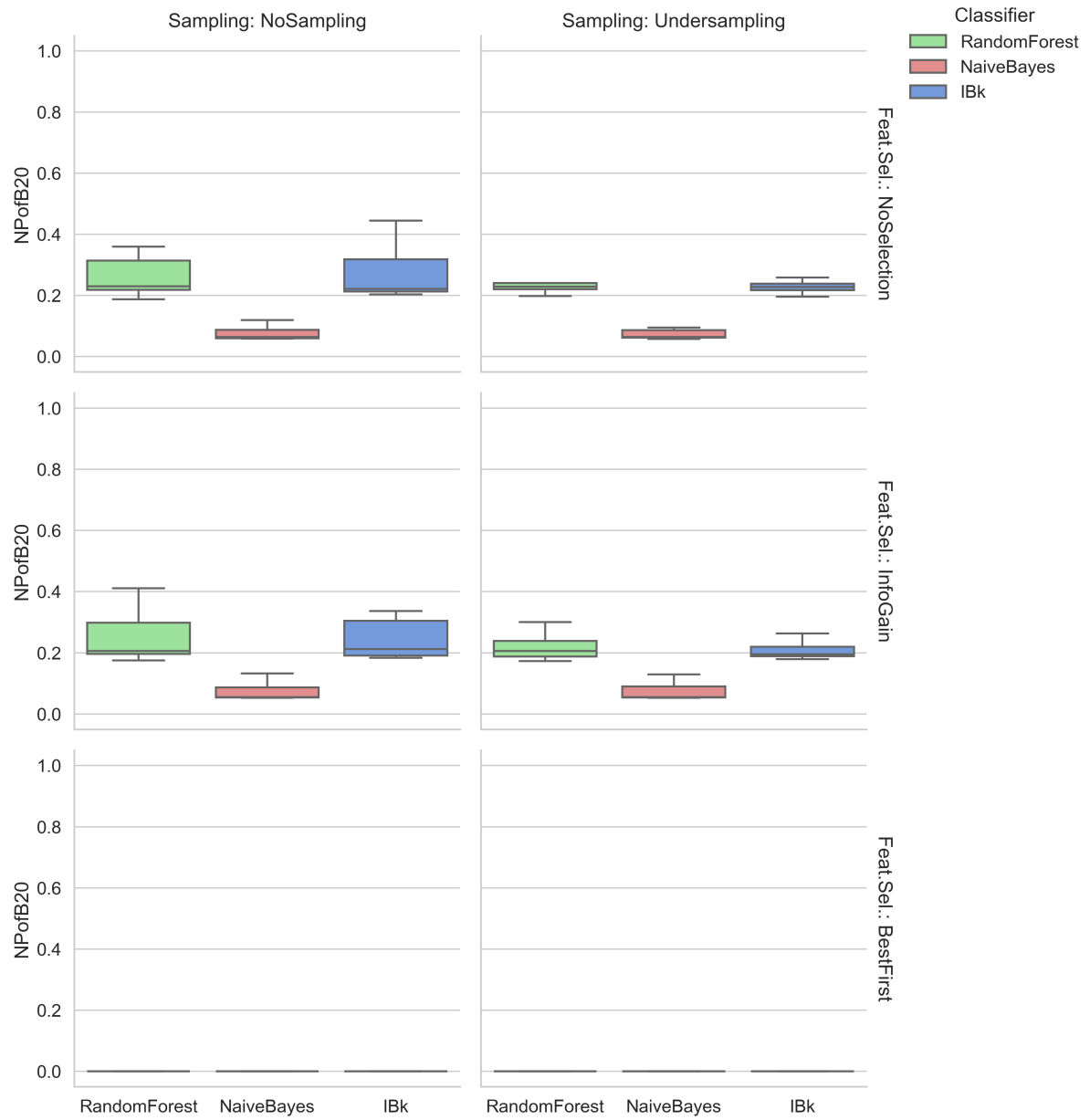


Tabella 7: Correlazione Metriche vs Bugginess OpenJPA

Metrica	Pea.	Spear.
LOC	0.219	0.234
NumLocalVars	0.198	0.226
NumStatements	0.200	0.220
NestingDepth	0.206	0.210
Cyclomatic	0.173	0.204
CognitiveCplx	0.195	0.201
Global_NAuth	0.239	0.193
Global_NR	0.190	0.188
Glob_LOC_Add	0.022	0.181
Glob_MAX_Add	0.014	0.174
Glob_AVG_Add	0.003	0.171
Glob_Churn	0.027	0.169
Glob_LOC_Del	0.034	0.163
Glob_MAX_Churn	0.019	0.161
Glob_MAX_Del	0.025	0.155
Glob_AVG_Churn	0.009	0.153
Glob_AVG_Del	0.021	0.145
AVG_LOC_Added	0.013	0.134
LOC_Added	0.008	0.134
MAX_LOC_Added	0.014	0.134
NAuth	0.138	0.132
NR	0.097	0.132
Churn	0.012	0.132
MAX_Churn	0.016	0.132
AVG_Churn	0.017	0.131
CodeSmells	0.144	0.115
LOC_Deleted	0.018	0.115
MAX_LOC_Del	0.017	0.115
AVG_LOC_Del	0.021	0.115
NumParams	0.043	0.040
ReturnCplx	0.036	0.036

Tabella 8: Dettaglio Metodo Pre-Refactoring OpenJPA

Metrica	Valore
Info	
Release	1.2.2
Signature	parseMember...
Statiche	
LOC	185.0
Statements	4.0
Params	1.0
Vars	11.0
Cyclomatic	57.0
Cognitive	4.0
Nesting	2.0
Return	1.0
Smells	2.0
Locali	
NR	3
NAuth	1
Added	6.0
MAX_Add	2.0
AVG_Add	2.0
Deleted	6.0
MAX_Del	2.0
AVG_Del	2.0
Churn	12.0
MAX_Churn	4.0
AVG_Churn	4.0
Globali	
GLNR	15
GLNAuth	4
GLAdd	97.0
GLMAX_A	42.0
GLAVG_A	6.47
GLDel	9.0
GLMAX_D	2.0
GLAVG_D	0.60
GLChurn	106.0
GLMAX_C	42.0
GLAVG_C	7.07
Target Attuale	
Bugginess	TRUE

Tabella 9: Dettaglio Completo Metriche Post-Refactoring OpenJPA (15 Metodi)

Metric	M1	M2	M3	M4	M5	M6	M7	M8	M9	10	11	12	13	14	15
Sig.	Key	Bsc	Idn	ElH	Mem	KyE	ELA	KyA	Str	MpK	Hnd	XEm	Joi	Std	Ovr
Metriche Statiche															
LOC	4	30	24	3	22	3	38	26	18	21	0	3	27	18	18
Stmt	1	1	1	1	3	1	1	1	1	1	0	1	1	1	1
Params	3	3	4	2	1	2	3	3	3	3	4	2	3	3	3
Vars	0	0	0	0	6	0	2	2	0	0	0	0	0	0	0
Cyclo	1	9	7	1	6	1	11	7	5	6	1	1	8	5	5
Cogn	0	1	1	0	7	0	1	1	1	1	0	0	1	1	1
Nest	0	1	1	0	3	0	1	1	1	1	0	0	1	1	1
Return	1	0	0	1	1	1	1	0	0	0	1	1	0	0	0
Smells	0	1	1	0	0	0	1	1	0	0	0	0	1	0	0
Metriche Processo (Locali)															
NR	1	1	1	1	4	1	1	1	1	1	1	1	1	1	1
NAuth	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Add	4	30	24	3	6	3	38	26	18	21	0	3	27	18	18
MaxA	4	30	24	3	2	3	38	26	18	21	0	3	27	18	18
AvgA	4	30	24	3	1.5	3	38	26	18	21	0	3	27	18	18
Del	0	0	0	0	169	0	0	0	0	0	0	0	0	0	0
MaxD	0	0	0	0	163	0	0	0	0	0	0	0	0	0	0
AvgD	0	0	0	0	42.3	0	0	0	0	0	0	0	0	0	0
Churn	4	30	24	3	175	3	38	26	18	21	0	3	27	18	18
MaxC	4	30	24	3	163	3	38	26	18	21	0	3	27	18	18
AvgC	4	30	24	3	43.8	3	38	26	18	21	0	3	27	18	18
Metriche Processo (Globali)															
G_NR	1	1	1	1	16	1	1	1	1	1	1	1	1	1	1
G_NA	1	1	1	1	4	1	1	1	1	1	1	1	1	1	1
G_Add	4	30	24	3	97	3	38	26	18	21	0	3	27	18	18
G_MxA	4	30	24	3	42	3	38	26	18	21	0	3	27	18	18
G_AvA	4	30	24	3	6.1	3	38	26	18	21	0	3	27	18	18
G_Del	0	0	0	0	172	0	0	0	0	0	0	0	0	0	0
G_MxD	0	0	0	0	163	0	0	0	0	0	0	0	0	0	0
G_AvD	0	0	0	0	10.8	0	0	0	0	0	0	0	0	0	0
G_Ch	4	30	24	3	269	3	38	26	18	21	0	3	27	18	18
G_MxC	4	30	24	3	163	3	38	26	18	21	0	3	27	18	18
G_AvC	4	30	24	3	16.8	3	38	26	18	21	0	3	27	18	18
Predizione del Modello															
Prob.	30	30	42	0.5	90	0.5	44	47	40	36	33	0.5	39	40	40
Risk	LO	LO	LO	LO	HI	LO	LO	LO	LO	LO	LO	LO	LO	LO	LO

Legenda: M1:parseKeyAnnotations, M2:parseBasicColumnAnnotations, M3:parseIdentityAnnotations, M4:parseElementEmbeddedMappingHelper, M5:parseMemberMappingAnnotations, M6:parseKeyEmbeddedMappingHelper, M7:parseElementAnnotations, M8:parseKeyAdvancedConfig, M9:parseStrategyAnnotations, M10:parseMapKeyAnnotations, M11:AnnotationHandler.handle, M12:parseXEmbeddedMappingHelper, M13:parseJoinAnnotations, M14:parseStandardKeyColumns, M15:parseOverrideAnnotations.

Tabella 10: What-If analysis openJPA no code smell

Dataset	Actual Bugs	Predicted Bugs (E)
Dataset A	53604	35733
Dataset B+	24752	20491
Dataset B (Fixed)	-	17696
Dataset C	28852	15242

Tabella 11: Risultati Aggregati per OpenJPA

Classifier	Sampling	Feat. Sel.	Prec.	Rec.	F-Meas.	AUC	Kappa	NPofB20
IBk	NoSampling	BestFirst	0.7474	0.1721	0.2722	0.6666	0.2105	0.0454
IBk	NoSampling	InfoGain	0.8366	0.3337	0.4736	0.7631	0.3967	0.2628
IBk	NoSampling	NoSelection	0.8113	0.3770	0.5109	0.7783	0.4315	0.2798
IBk	Undersampling	BestFirst	0.4007	0.5325	0.4476	0.6850	0.2687	0.0277
IBk	Undersampling	InfoGain	0.5073	0.5798	0.5290	0.7641	0.3924	0.2128
IBk	Undersampling	NoSelection	0.5357	0.5979	0.5445	0.7802	0.4176	0.2359
NaiveBayes	NoSampling	BestFirst	0.5063	0.2430	0.3218	0.6595	0.2126	0.0167
NaiveBayes	NoSampling	InfoGain	0.4904	0.2334	0.3060	0.6543	0.1990	0.0727
NaiveBayes	NoSampling	NoSelection	0.4933	0.2089	0.2758	0.6521	0.1806	0.0789
NaiveBayes	Undersampling	BestFirst	0.4444	0.3349	0.3720	0.6637	0.2307	0.0163
NaiveBayes	Undersampling	InfoGain	0.4747	0.2669	0.3279	0.6568	0.2108	0.0725
NaiveBayes	Undersampling	NoSelection	0.4864	0.2109	0.2783	0.6560	0.1795	0.0796
RandomForest	NoSampling	BestFirst	0.7708	0.1807	0.2856	0.7005	0.2230	0.0447
RandomForest	NoSampling	InfoGain	0.8647	0.3428	0.4864	0.7999	0.4095	0.2619
RandomForest	NoSampling	NoSelection	0.8642	0.3822	0.5226	0.8309	0.4461	0.2708
RandomForest	Undersampling	BestFirst	0.3999	0.5500	0.4522	0.6981	0.2720	0.0310
RandomForest	Undersampling	InfoGain	0.4985	0.6082	0.5322	0.7835	0.3923	0.2282
RandomForest	Undersampling	NoSelection	0.5299	0.6248	0.5529	0.8123	0.4244	0.2516